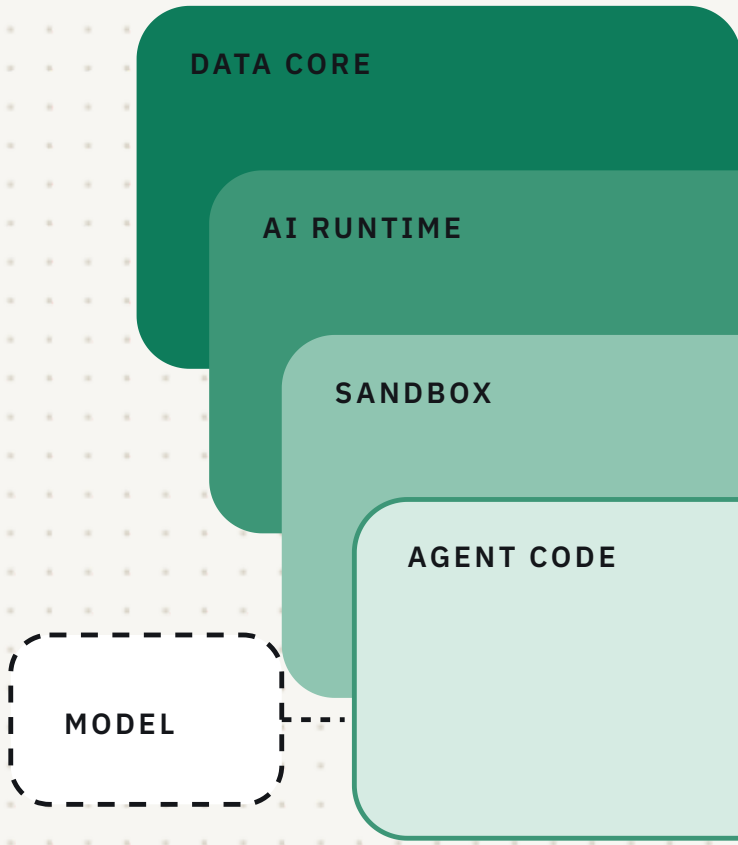
DecentAI



TECHNICAL PAPER · EDITION 1 · OCTOBER 2026

An operating system for AI agents, in which code decides and the model proposes.

The architecture, protocols and mechanisms of DecentAI: identity and policy, the gate every call passes, agents as declared contracts, confined execution, secrets that never reach the model, evidence, and how it is deployed and verified.

Ahmed Dirir · DecentAI · ahmed@decentai.io · decentai.io

ABSTRACT

The model may propose anything. It decides nothing.

Language models can now plan multi-step work and call tools, so they can act on an organization's mail, documents, records and systems. They are unreliable in two ways that matter for action: they state false things fluently, and they follow instructions hidden in whatever they read. The tools they call are programs written by third parties. A platform that lets such a planner act must assume that the planner will sometimes be wrong or hostile, and that the code it drives may be hostile too.

DecentAI is a self-hosted platform built on that assumption. It separates *proposal* from *authority*: the model's output is a request, and code the model cannot read or influence decides every request. A deny-by-default policy is checked on each call. A gate compares each function's declared cost with the trust the person gave the chat, and holds anything costlier until that person approves the exact inputs. Approved agent code runs as its own operating-system user, in a kernel-confined sandbox whose only way out is a proxy open to the hosts the agent declared. Credentials go from an encrypted vault to the agent's code for one call, never to the model. What the interface shows as data, or as a finished step, comes from recorded results, not from the model's prose.

This paper describes the whole system for engineers who will run, extend or audit it. Settings and limits are named as the platform names them; where a property depends on the host, the paper says so.

1 **Principles and threat model**
What is trusted, with what; the decisions every other chapter follows from.

2 **Architecture**
The engine's parts, containers and networks; where state lives.

3 **Anatomy of a turn**
One message, end to end, as a sequence of checked steps.

4 **Identity, tenancy and authorization**
Organizations, groups, policies, sessions, ownership and sharing.

5 **The assistant**
Context, the step loop, plans and verification, helpers, memory.

6 **The gate**
Permission levels, trust levels, approval cards and Safety rules.

7 **Agents**
The manifest as a contract; sources, review, approval and grants.

8 **Execution nodes and the sandbox**
Linking a node, workers, confinement, the proxy, the live screen.

9 **Secrets and data**
The vault, credentials, connected apps, records, files and storage.

10 **Evidence, monitoring and audit**
What a call leaves behind, and who can see it.

11 **Time and reach**
Schedules, webhooks in and out, notifications, stopping.

12 **Clients**
The web client, the phone app and the desktop app.

13 **Deployment**
One host, a separate node, two VMs on AWS, a desktop install.

14 **Verification**
Test suites, documentation checks, and the evaluation.

15 **Limits and open questions**
What the platform does not do or guarantee.

A **Glossary**
The vocabulary, in one place.

Sections are numbered within chapters (3.2), as are figures (Figure 3.2). Properties the system holds are stated once as numbered **invariants** (I-6.1) in a green margin, so that a reader can find them, test them and hold the code to them. Diagrams use one colour language throughout: ■ the platform's own parts, ▢ an agent's code, ⬚ the model, ▢ a step that waits for a person, ▢ refused or stopped. What the paper describes is the engine, its three clients, its deployment recipes and its catalogue of agents, as released in October 2026.

CHAPTER 1

Principles and threat model

Every mechanism in DecentAI follows from a short list of assumptions about what can go wrong and a shorter list of decisions about where authority lives. This chapter states both, so that the rest of the paper can be read as their consequences.

1.1 What is assumed to go wrong

The platform is designed against four kinds of failure, each assumed to happen in normal operation rather than as a rare attack.

WHO OR WHAT	HOW IT FAILS	WHAT THE DESIGN DOES ABOUT IT
The model	States false things fluently; follows instructions hidden in mail, pages and files it reads (prompt injection); proposes calls that are wrong, wasteful or harmful.	Holds no authority, no credential and no secret. Its output is a proposal that passes checks which do not read its prose (chapters 5–6).
An agent's code	Is written by a third party; may be buggy or hostile; may try to read other agents' files, reach hosts it did not declare, or escape its process.	Runs as its own user, confined by the kernel, with a proxy as its only way out, open to the hosts its manifest declared (chapters 7–8).
Content	A message, attachment, web page or tool result carries text written to steer the assistant.	Content is data. It can change what the model proposes, never what the platform permits.
A person	Approves without reading; holds more access than a task needs; leaves an organization.	Approvals show the exact inputs and bind to them; grants are narrow and revocable; ownership hands over when its holder leaves (chapters 4, 6).

Two parties are trusted: the operator, who controls the deployment's hosts and keys, and the platform's own code. Nothing here protects against an operator with root on the host, or against a compromised Data Core. The aim is that a compromise of any *other* part — the runtime, the sandbox, an agent, the model, a client — does not become a compromise of the whole.

1.2 Where authority lives

The central decision is to put the model outside the chain of authority. In a conventional assistant the model's tool call *is* the action, guarded at best by a confirmation the model can describe in its own words. In DecentAI the tool call is a proposal from an unprivileged party. Code decides it from facts the model does not control: who the person is, the organization's policy, the agent's approved manifest, the chat's trust level and, when the call costs more than that trust, the person's yes to those exact inputs.

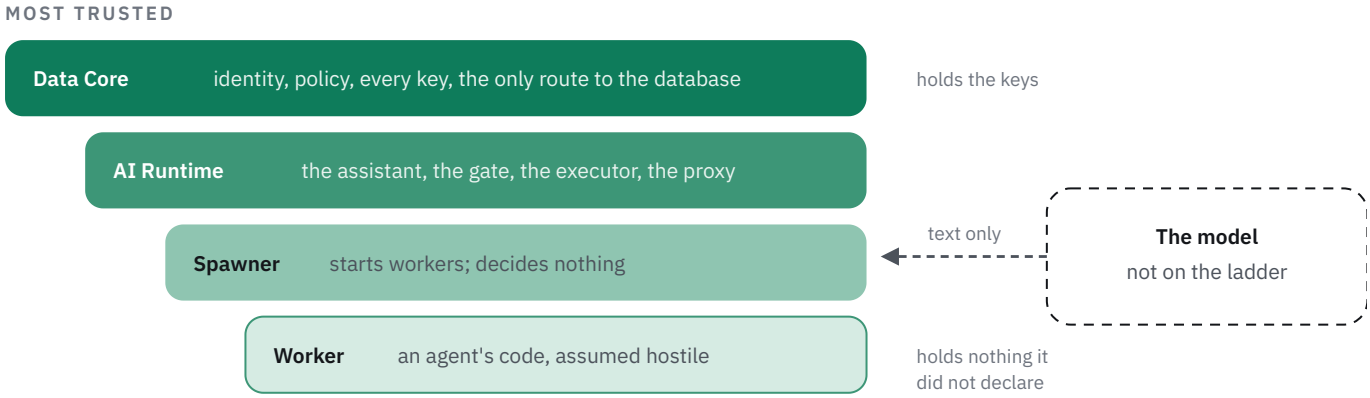


Figure 1.1 — The ladder of trust. Each part is trusted with what its job needs and nothing more, so taking one over gains as little as possible. The model is not a rung: it reaches the runtime as text, and nothing it writes is read as permission.

1.3 The decisions

Six decisions shape the implementation. Each is enforced in one place, so that it can be read, tested and argued about there.

Deny by default

No action is permitted unless a policy chain grants it to the person, walked on every request in the Data Core's one authorization point. A page that forgets to declare its action is refused, not opened.

One door

Clients, nodes and webhooks reach the engine through the Data Core alone. The runtime has no database and no route to one; it asks for what it keeps.

Contracts before code

An agent declares its functions, their cost, what it keeps and where it connects before anyone approves it. The executor holds it to that declaration on every call.

Decisions bound to inputs

A person's yes covers one call with one set of inputs, identified by a hash. Changing a character asks again.

Evidence over prose

Tables, files and "done" marks shown beside the model's words come from stored results of calls that ran. A claim with nothing behind it is shown as a claim.

Limits stated

Where a protection depends on the host — a kernel feature, a container runtime — the platform checks for it, refuses to start without the essential ones, and reports what is not enforced.

I-1.1 No model output is an authorization. A function runs only when the person's policy, the agent's approval and grant, the chat's enabled set and the gate all permit it, each decided by platform code from stored state.

I-1.2 No secret is ever placed in the model's context. Credentials travel from the vault to the agent's worker for one call (chapter 9).

I-1.3 The audit trail is append-only from the platform's side: no code path in the engine updates or deletes an audit row.

1.4 Why an operating system

The problem has a familiar shape. An operating system runs programs written by strangers, gives each a user of its own, passes every request through a system call it may refuse, and keeps a log. DecentAI applies the same ideas one level up.

IN AN OPERATING SYSTEM	IN DECENTAI
a program, and the permissions it asks for	an **agent**, and its **manifest**, reviewed before approval
a process, as its own user	a **worker**: one agent's code, as that agent's own user
a system call the kernel may refuse	a **function call**, decided by the gate
the firewall	the **proxy**, open only to the hosts the manifest declared
"Allow this app to…?"	an **approval card**, bound to one call
the scheduler, and the system log	the **clock**, and the **audit trail**

DecentAI is not a kernel. It runs on Linux in containers and uses the kernel's own protections for the parts that must be confined. What it adds is a place for a planner that is capable and not trusted.

CHAPTER 2

Architecture

The engine is three programs — the Data Core, the AI Runtime and the Execution Sandbox — and the clients that speak to it. This chapter names them, shows what runs and what speaks to what, and says where every piece of state is kept.

2.1 The parts

PART	SERVICE	WHAT IT HOLDS AND DOES
Data Core	`data-core`	Identity and access, the agent lifecycle, the data layer, chats and approvals, organization settings, execution nodes and the clock. The only part that reaches the database; the engine's one published port.
AI Runtime	`ai-runtime`	One assistant per chat, the executor every call passes and its gates, what a schedule's fire runs, and the egress proxy agents reach the internet through.
Execution Sandbox	`sandbox`	Agent code: a spawner that starts workers, and one worker per call, each as the agent's own user.
Execution node	—	One AI Runtime with its sandboxes, deployed together. The bundled install has one; more can be linked from other machines (chapter 8).
Clients	released on their own	The web client, the phone app and the desktop app. Each is told the engine's address and speaks only to the Data Core, across origins, with a session token (chapter 12).

The three programs together are the **AI Engine**. The engine has no user interface: every client reaches it over the same HTTP and WebSocket API. So one engine serves a browser, a phone and a desktop window, and each client ships on its own schedule.

2.2 What runs

The bundled install starts seven containers; six stay up and a seeder runs once at every start. Figure 2.1 shows them with the connections between them; every arrow is a connection its source opens.

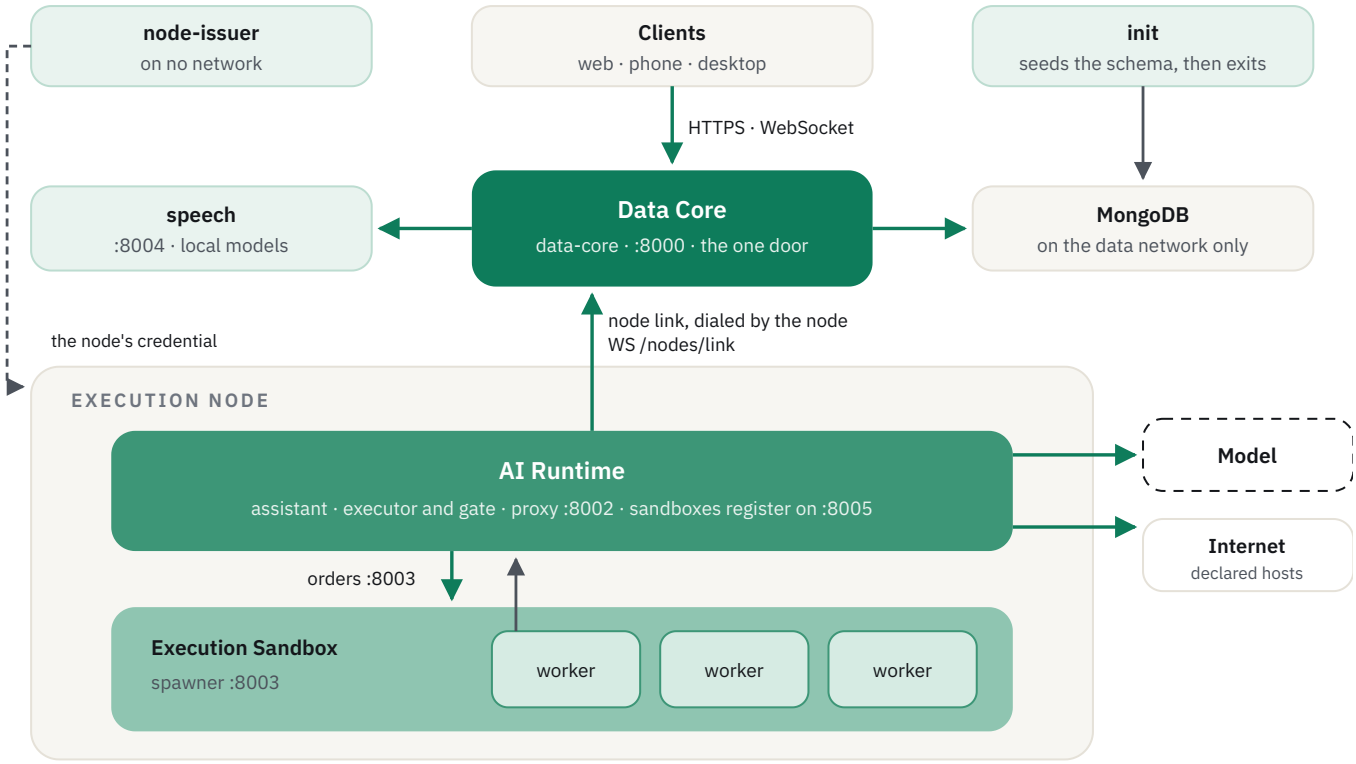


Figure 2.1 — The bundled install. The Data Core is the only published port and the only part that reaches the database. The execution node dials the Data Core's node link; the runtime orders the sandbox's spawner, and the sandbox registers with the runtime, each side proving the node's secret. Workers reach the internet only through the runtime's proxy, to the hosts their agent declared. The node issuer rewrites the bundled node's credential every few minutes from the install's authority key.

2.3 Networks

Three container networks carry the connections, and two of them are `internal: true`: they have no way out.

NETWORK	ON IT	PURPOSE
`default`	`data-core`, `ai-runtime`, `speech`	The one with a way out: clients reach the Data Core's published port; the Data Core reaches the speech service and the internet (repositories, mail, OAuth, outbound notifications); the runtime reaches the model's provider and, for agents, the internet through its proxy.
`data`	`mongo`, `data-core`, `init`	Internal. The database and the two that speak to it; nothing else has a route to it.
`sandbox`	`ai-runtime`, `sandbox`	Internal. The runtime is the only container on it with a way anywhere, so a worker's only way out is the proxy.

> **I-2.1** Only the Data Core holds a database credential or a route to the database. Everything the runtime keeps, it keeps by asking the Data Core, under the person's delegation for that chat.

> **I-2.2** An agent's worker has no network route except the runtime's proxy, and the proxy forwards only to hosts the agent's approved manifest declared.

2.4 Where state lives

The runtime and the sandbox are replaceable: a node can be restarted, drained or rebuilt without losing a chat. Durable state lives in the database and a few volumes.

KEPT IN	WHAT
MongoDB	Every record of the platform: organizations, people, groups and policies; agents' approvals, grants and records; chats, messages, events and approvals; credentials (sealed); schedules, webhooks, notifications and the audit trail — 45 collections, all reached through org-scoped stores in the Data Core.
Uploads volume	Files, unless the organization stores them in S3, Azure Blob or Google Cloud Storage instead.
Agent packages	The approved code of each agent, by digest; nodes pull it from the Data Core.
Node volumes	Prepared agent environments, the sandbox's working folders and the node's credential: rebuildable from the above.
Speech models	Pinned model files, fetched once and checked against their published digests.

A chat in progress is held by one runtime at a time under a lease kept in the Data Core. If that runtime goes away, the lease lapses and the next one hydrates the session from the stored state — including a call parked for approval, which is saved before its card is shown (chapter 6).

Speech is the phone's or computer's own by default, or the organization's: a provider it named, or the platform's own models in the `speech` container (Whisper to write speech down, Piper voices to say answers). The container holds no key and no setting, speaks the providers' audio protocol so the choice between them is an address, and keeps nothing it is sent.

CHAPTER 3

Anatomy of a turn

One chat message, followed through every process it crosses, from the person's socket to a worker in a sandbox and back. Each step names what carries it and what could stop it.

A turn is not a request and a response. Each chat has one assistant with durable state, and a message is one more event for it. The assistant advances in **beats**: one model call, one action, one observation, one save. A turn ends when a beat says it is finished, when the work waits on a person, or after 20 beats by default. The sections follow the steps of Figure 3.1.

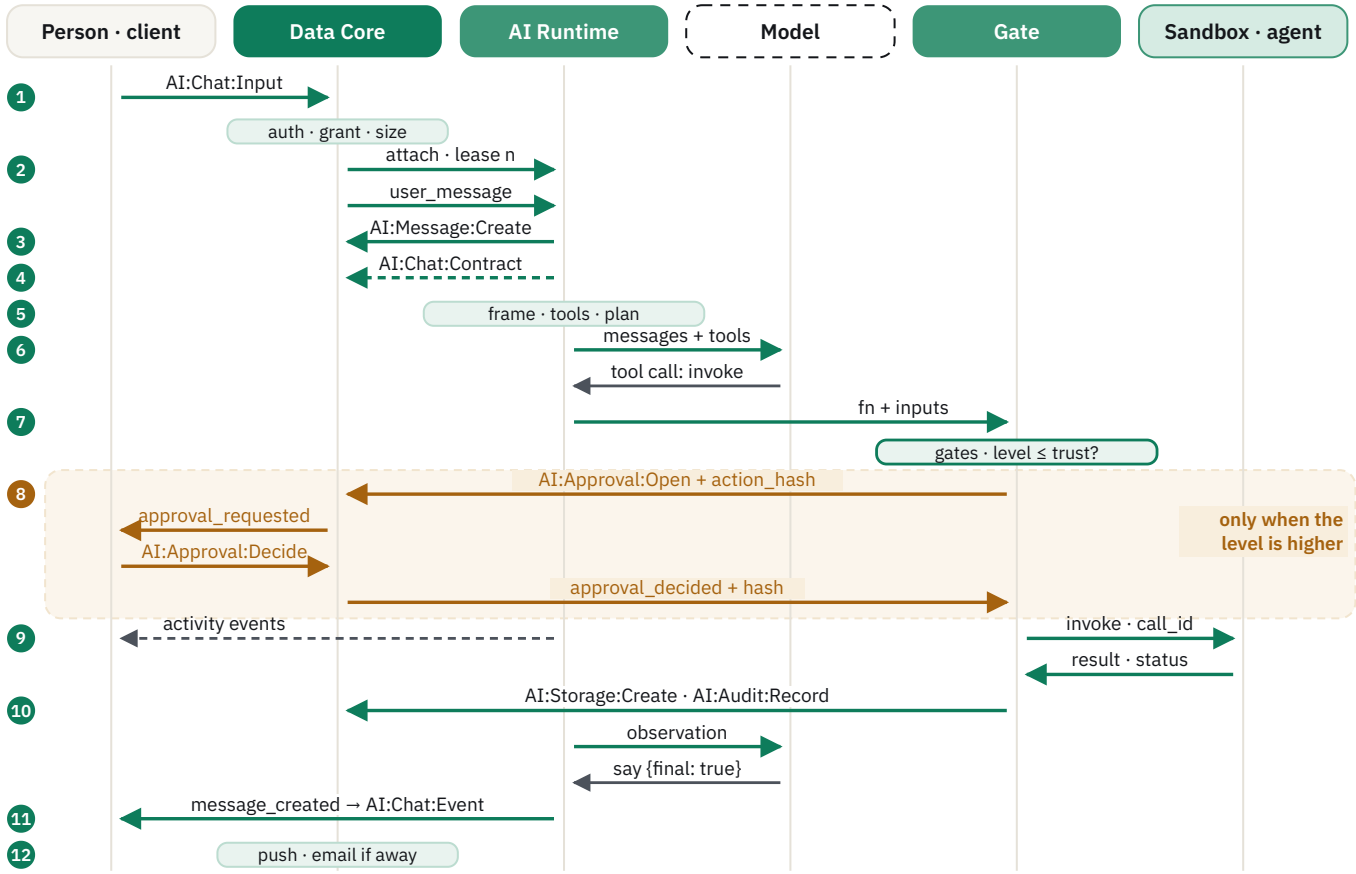


Figure 3.1 · One turn, one gated call. Green arrows carry the work; ink arrows come from the model or relay news. The amber band happens only when the function's level is above the chat's trust level. The Gate is the runtime's executor, drawn apart because nothing reaches the sandbox except through it.

3.1 The message enters the Data Core

Step 1. The Data Core takes a message only from the chat's own person, with the right permission, within size limits. The web client keeps one WebSocket per open chat. A browser cannot set headers on a WebSocket handshake, and a token in a URL ends up in proxy logs, so the client sends its session token as a subprotocol. The socket closes unless the caller is a person and the chat is theirs, in this organization and not archived.

Socket frames bypass the `/app` gateway, so the socket checks each frame's action itself: `AI:Chat:Input` requires `ai:chat:sendmessage`, with at most 256 KB of text and 20 attached files. A refused frame stores nothing, so the transcript never holds words the assistant was not given.

3.2 Placement, lease and delegation

Step 2. Before forwarding, the Data Core's relay makes sure one execution node holds the chat. Placement keeps the node with the chat's live lease. Failing that, it prefers the organization's own nodes, then the chat's last node, then the least loaded. The relay then issues two things:

Lease	Node, number and expiry. Heartbeats renew it every 20 seconds for 60. Every call for the chat carries the number, and the gateway accepts only the live one from its own node, so two nodes never both act for one chat.
Delegation	A token for one person, one chat and one node. It lives an hour, is renewed after 45 minutes, and counts only beside that node's own credential.

Both travel in a signed `attach` frame, followed by the message. The delegation says *who* is acting, never *what* they may do. A node runs agents' code, so it holds nothing that outlives the chat.

3.3 The session hears the message

Step 3. On the node, a message is stored before the assistant sees it. The session, live or rebuilt from saved state, stores the message (`AI:Message:Create`); a resend finds the first copy and stops. The session then emits `message_created`, runs the pre-thinking hook and records the inbox event. A process that dies mid-beat loses nothing: the next one replays the inbox.

Step 4. The hook re-reads the chat's **contract** (`AI:Chat:Contract`), the only message from the Data Core to the runtime that carries authority. It names the agents installed, granted and enabled in this chat, each pinned by package digest and manifest hash. It also carries the function grants, the trust level, the model, the turn budget, the skills, the organization's rules and the person's MCP servers. Because it is re-read every turn, a grant withdrawn this morning is gone by the next message, with nothing pushed to the runtime. An agent the node lacks is fetched by digest while the chat's open pages (its **audience**) see `agent_status`.

3.4 What the model is shown

Step 5. The runtime puts what changes least at the front of each request, because a provider's prompt cache reuses the front (Figure 3.2). The tools are the 20 action tools plus one per function of each agent the model has *opened*, at most 128 in all, the most OpenAI-compatible providers accept. Each carries the manifest's input schema, which steers the model; the executor still validates every input.

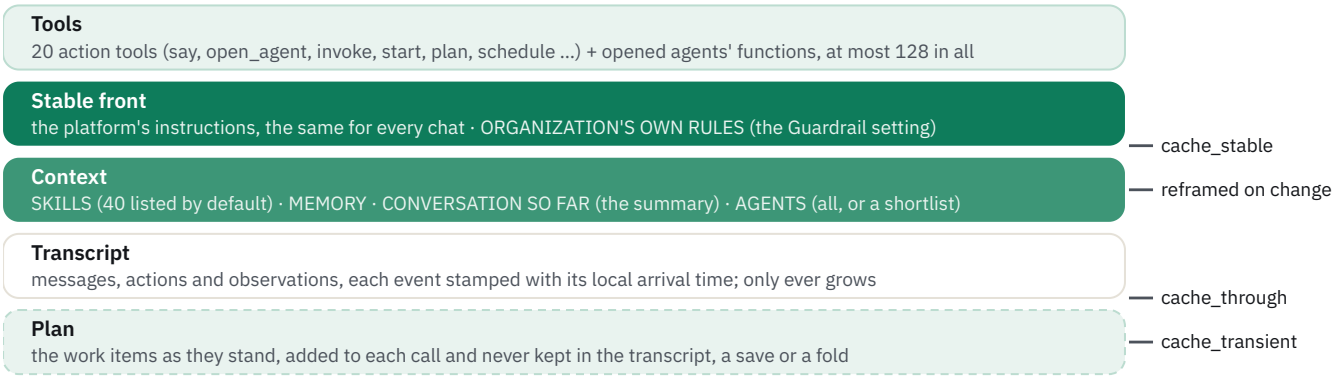


Figure 3.2 · The request one beat sends, front to end. The Anthropic, Vertex and Bedrock connectors turn the cache hints into cache marks; OpenAI and Gemini cache repeated prefixes on their own.

With up to 15 agents, AGENTS lists them all. Beyond that, a router embeds the person's words, keeps 50 candidates, may rerank them with the chat's model, and lists 15 plus any already open. A skill's body loads only on `use_skill`. Long transcripts are folded into the summary, and folded lines stay searchable through `recall`.

3.5 One model call, one action

Step 6. Each beat makes one model call and takes back exactly one action, as a tool call or as strict JSON. A call to a function's own tool is rewritten to `invoke {function, inputs}` before anything reads it, so the gate, the trace and the evidence share one vocabulary. A reply that does not parse is sent back; one cut off at the output limit is never acted on.

3.6 The gate

Step 7. Every proposed call passes the executor's gates in a fixed order. None reads what the model wrote around the call. They check that:

1. the function is declared in the approved manifest;
2. the chat's grants reach it;
3. reference inputs resolve to stored results, and declared defaults are applied;
4. the inputs validate against the manifest's schema;
5. each scope the call acts on is a single named value within the grant's constraints;
6. the function's permission level is at or below the chat's trust level.

Step 8. When the level is higher, the call is parked durably before its card goes out. `AI:Approval:Open` records the call with its **action hash**: SHA-256 over key-sorted JSON of `{agent_id, function, inputs}`. Only a person decides: `AI:Approval:Decide` is outside the 41 actions a delegation may call (§4.3). The decision is recorded before the relay sends it on with the stored hash.

Before a yes is delivered, the contract is read again, so a grant narrowed while the card waited refuses the call. A call rebuilt after a restart must hash to the stored value, or it is blocked. A Deny also bars the hosts the call named from every function of that agent for the rest of the request, so a function that needs no card cannot make the same visit.

> **I-3.1** No call runs above the chat's trust level unless a person approved that exact `(agent, function, inputs)` triple. Its hash is recomputed before dispatch, and the grants are re-read after the approval.

3.7 Dispatch to a worker

Step 9. The worker that runs the agent's code holds no credentials. Past the gates, the executor builds a **call object**, the invocation's whole authority: exactly the resource operations the function declared, and a model only if the manifest says `llm: true`. The timeout is the manifest's, or 60 seconds.

The worker pool finds or starts the agent's worker in an Execution Sandbox that has proved its isolation locks. The worker runs as the agent's own user, confined to its own files, with the egress proxy as its only route out. The call is one line of JSON: `invoke {call_id, function, inputs, conversation}`. Each resource the worker needs is a request back naming its `call_id`, refused unless the function declared it.

3.8 Result, evidence and activity

Step 10. The worker's `{result, status}` is a claim, not a fact. The host strips the keys only the platform may write, validates a success against the output schema, and stores it (`AI:Storage:Create`). Every call, whatever its outcome, gets one line on the audit trail (`AI:Audit:Record`), and the audience hears its progress as `activity` events.

The model sees a result whole up to about 16,000 characters, or else a 12,000-character preview with its `storage_ref`. **Evidence** comes from the trace, never from prose: a table or chart appears only when `say.show` names a successful traced call, and its rows are rendered from storage. Writes (level above 0) ride on the message as `success` parts.

3.9 The answer, and the person who is not looking

Step 11. The next beat usually ends with `say {text, final: true}`. The runtime stores the message and emits `message_created`, first to the durable log, then to the page. Answers are not streamed token by token; a turn streams as events, from `working` to `idle`. A reconnecting page replays what it missed from the log.

Step 12. When a card opens or the assistant answers while none of the person's browsers has the chat open, the Data Core pushes to every subscribed device. It emails only if no device was reached, at most once per chat every ten minutes. A notification names the agent and step, or only that something happened if the person or organization prefers.

> **I-3.2** Every event an audience hears that has a replay was recorded before it was sent. A socket is an audience, not a lifeline: detaching stops the live relay, never the work.

Events the client receives

The list is defined once, in a protocol module the runtime and the Data Core share.

EVENT	MEANING	RECORDED
`hello`	on attach: the present state, active jobs, pending cards, plan	socket only
`working` · `idle`	a cycle starts advancing · comes to rest	yes
`message_created`	a stored message: the person's echo, the assistant's answer, an agent's post	yes
`activity`	calls, jobs and helpers starting, progressing and finishing	yes
`plan_updated` · `memory_saved`	the work items changed · a fact was remembered	yes
`approval_requested`	a call waits for the person: agent, function, level, inputs	yes
`question_asked` · `question_closed`	a card asks for words, files, code, a login or a skill · it was answered or expired	yes
`schedule_set` · `schedule_removed` · `sleeping`	the clock changed; the assistant paused until a time	yes
`stopped`	the kill switch ended the work	yes
`agent_status` · `screen_frame`	an agent is being installed · a picture of a browser an agent drives	socket only
`runtime_unavailable` · `invalid_input` · `work_stopped`	from the relay: no node may serve the chat, the input was refused, the person stopped everything	relay

CHAPTER 4

Identity, tenancy and authorization

An identity is a row, never a bundle of permissions. What the row may do is found on every request, by walking a chain the organization keeps.

4.1 Organizations, users and groups

The **organization** is the boundary between tenants. Every row that belongs to it carries its `org_id`; authorization never reads the organization's name, which can change. A **user** belongs to one organization, and their email is unique across the deployment. A user's access is exactly their group membership. There is no administrator flag and no permission on a user directly, so "why may this person do that" is always a path that can be followed and changed.

Each organization has a built-in **Everyone** group that every user is in implicitly. A new organization is seeded with three chains:

POLICY	ROLE	GROUP	WHAT IT CARRIES
`FullAccess`	Administrator	Administrators	the wildcard, including every escape grant
`BaseAccess`	User	Everyone	the 69 baseline actions: one's own profile, memories, API keys, schedules, MCP servers, notifications. Chatting is not among them.
`MemberAccess`	Member	Members	37 everyday actions: `ai:chat:*` and the rest of one's own chats, files, records and credentials, and seeing the agents one was given

The baseline is versioned: a new baseline action is offered to each policy once, so a restart never undoes an administrator's deliberate removal.

4.2 Roles, policies and action strings

A **role** bundles policies and reaches a person only through a group. A **policy** is a list of statements:

```
{"effect": "Allow", "actions": ["agents:agent:install", "iam:group:*"], "resources": ["*"]}
```

An action is an endpoint. `POST /app {"endpoint": "Agents:Agent:Install"}` is checked as `agents:agent:install`, so there is no second permission vocabulary to keep in step with the API. The vocabulary is the **action catalog**: 224 actions in 43 services. It is part of the reviewed code, not a file a deployment can edit, because every entry is a security boundary, and a pattern that matches no real action is refused. Resources are always `"*"`; who sees an individual document is a question for sharing (§4.7).

An installed agent's functions are deliberately *not* in the catalog: they come and go with manifests, and policies naming them would go stale. They are granted by **agent grants** instead: allow-only rows keyed by the agent, each naming groups and users, functions or `*`, and optional constraints. A bare `*` in a policy grants no agent function.

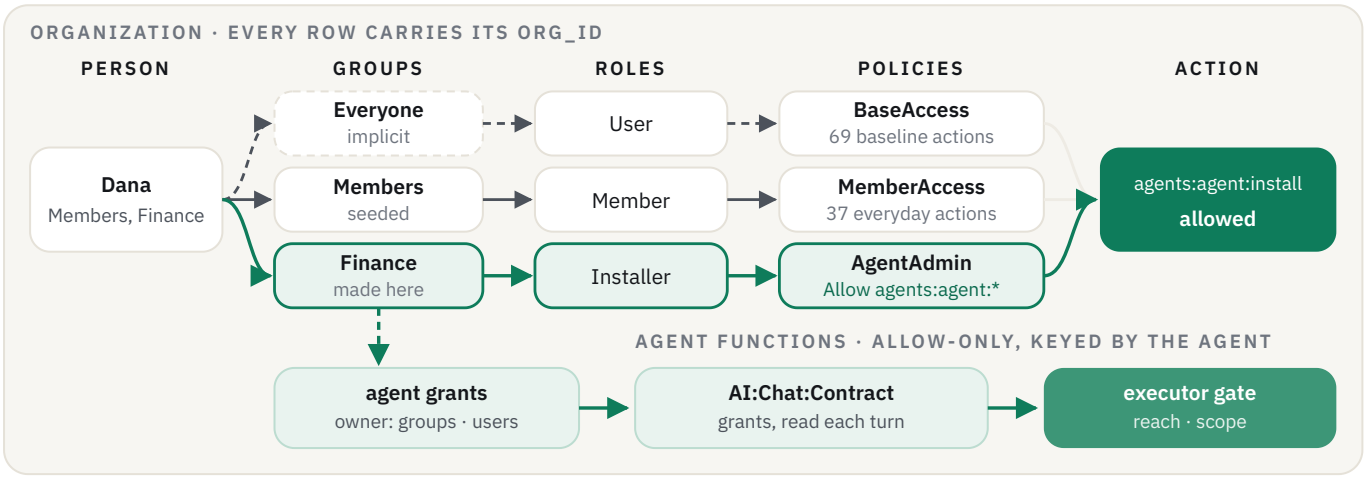


Figure 4.1 · How a grant reaches a person. Top: the IAM chain, walked on every request. Only the Finance path allows `agents:agent:install`; a matching Deny anywhere in the chain would override it. Bottom: an agent's functions need a separate, allow-only grant, which reaches the gate only through the chat's contract.

4.3 One decision, in one place

Every endpoint is authorized once, in the gateway's router; controllers hold no authorization code. The router asks the access controller, then also requires the action to be in the catalog, so a wildcard cannot reach a helper a controller happens to expose. The decision depends on the caller (Figure 4.2):

- A **node** may call the three package-door actions and nothing else.
- A **runtime** delegation is first held to a fixed list of 41 action patterns, so even an administrator's delegation cannot reach identity, account settings or approval decisions. Each chat-bound call must also name the delegation's own chat.
- For the person, the policy engine gathers every statement reachable through their groups. A matching `Deny` refuses; otherwise a matching `Allow` permits; no match refuses.

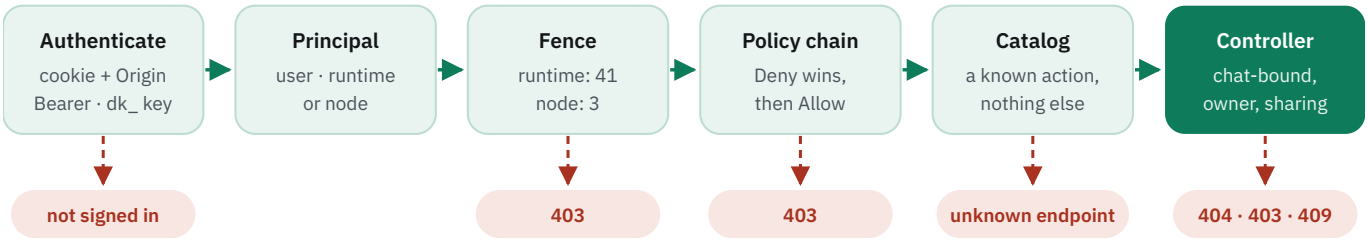


Figure 4.2 · The order of the checks on `POST /app`. The first check that fails is the answer. The controller checks only the record: is this chat the delegation's, may this person see or change this document.

Each process caches resolution for 60 seconds, but every write that could change a decision clears the cache. A revocation made through the application therefore counts from the next request; one made from the command line, within the minute.

4.4 Tokens and sessions

A session token is a pointer to a row, not a container of rights. Signing in writes a session row and returns a signed token that *names* it. Every request resolves the session and the user again, so signing out, disabling and deleting take effect at once. Sessions last 7 days, or 90 when the person asks to stay signed in.

CREDENTIAL	PRESENTED AS	NOTES
Browser session	`access_token` cookie	accepted only with the app's own `Origin`, because a browser sends a host's cookie to every port on it
Web or app session	`Authorization: Bearer …`; on the chat socket, the subprotocol	returned in the body at sign-in; a native app's session may list API keys but not make or revoke them
API key	`Bearer dk_…`	acts as the person; only its hash is stored; made and revoked only from a browser session, so a leaked key cannot make itself permanent
Runtime delegation	`Bearer` + node and lease headers	one person, chat and node, for an hour; held to the 41 runtime actions; survives sign-out, so a scheduled chat runs while its person is away, but not disabling or deleting
Node credential	`Bearer` on the node link	signed by a registered authority's private key; valid for minutes; opens only the package door

Passwords are hashed with bcrypt over a SHA-256 pre-digest, so bcrypt's 72-byte limit never truncates a passphrase. Sign-ins are throttled to 5 attempts per account and 20 per address in 15 minutes, checked before the password, so a locked window answers a right password exactly as a wrong one. Using a reset link ends every session and API key the person had.

> **I-4.1** No authority is copied into a token. Every credential names a row or a principal, and what that principal may do is read from the live chain, or for a chat from the live contract, at the moment it acts.

4.5 First run, setup code and invitations

Nobody signs themselves into an existing organization. An organization is created in one of three ways:

- **First run.** While the deployment has no users, `POST /auth/setup` creates the organization with the caller as administrator; two simultaneous submissions still produce one. Where the address is reachable from other computers, setup also asks for a **setup code**, which the deployment writes to its log unless the operator sets one or turns it off.
- **On the machine.** The operator's bootstrap command creates the organization and invites its first administrator.
- **Sign-up.** Only when the operator opens it and mail is configured. Nothing is created until the emailed link is followed.

An **invitation** stores only its token's hash and lasts seven days. The account's address and organization come from the invitation, never from the request, because the link proves one mailbox and nothing more. Its groups are given only if the inviter could still give them at acceptance.

4.6 Two invariants

> **I-4.2** An organization cannot lock itself out. Before any change to the access chain commits, it is checked that at least one *active* user of *this* organization still holds `iam:user:set_groups`, the recovery capability. If none would, the change is undone and refused.

I-4.3 Nobody grants what they do not hold. An author may *allow* only actions they hold themselves. This is checked when a policy is written and again at every attachment of a policy to a role, a role to a group or a group to a user, so escalation cannot be laundered through one. A `Deny` only narrows access, so anyone may write one.

4.7 Ownership and sharing

Seeing a document and changing it are separate rights. **Who may see** it is its owner map, a set of groups and users. **Who may change** it is its steward, the person recorded as its creator. One sharing engine serves every shareable type, because two copies would drift. Every read is filtered through it, so a document out of reach does not exist as far as the API can tell. Everything starts private, and a writer shares only into groups they belong to and with their peers. Seeing is never authority: a colleague can use a shared credential but cannot edit or delete it.

DOMAIN	SHARING	STEWARD AND ESCAPE GRANT
Secrets, files, records, skills	personal; organization-wide only with the escape grant	the creator; `…:set_owner_any` per domain, reaching only documents its holder can already see
LLM connections, agent sources	infrastructure: the creator may share with Everyone, and always stays in the map	`settings:llm:manage_any`, `agents:agent:source_manage_any`, reaching every one in the organization
Chats, memories, MCP servers, API keys	never shared	the person only; no escape grant
Installed agents, agent and credential grants	the organization's	whoever holds the action (`agents:agent:delete`, `…:grant`)

An agent's use of a credential is a grant, not a share: a revocable record per agent and slot. Stewardship can be transferred; the new steward takes the old one's place in the owner map, so every viewer keeps seeing the document.

4.8 Hand-over, and isolation between tenants

Deleting a user hands over what others rely on and removes what was only theirs. The deletion names a successor, by default the deleting administrator, and in one pass:

- drops sessions, keys, invitations and delegations;
- deletes chats with everything under them, memories, MCP servers and webhooks;
- passes stewardship of secrets, records, files, skills, model connections and agent sources to the successor, with every share intact;
- takes the person out of agent grants, and keeps the audit trail as recorded.

Disabling a user keeps everything but drops sessions, keys and delegations, closes live chats and pauses every schedule, so nothing acts unattended for someone who may not act.

Isolation lives in the queries, not in filtering afterwards. Organization-scoped stores put `org_id` beside every id a caller supplies, so another tenant's id finds nothing. Nodes keep the same boundary: placement never uses another organization's node, an organization may refuse shared nodes, and a node admitted by one organization hears only of its packages. A disabled organization keeps its data and refuses everyone in it; the application cannot delete an organization.

CHAPTER 5

The assistant

Each chat has one assistant, a saved record rather than a running process. A session loads it, feeds it events, lets the model choose one action at a time, and saves it after every step. The platform checks each choice, carries it out, and records only what actually happened.

5.1 One assistant per chat, kept as state

The assistant is one saved state object, so any process can continue where another stopped. A user message is not a fresh start, only one more event. The state holds the transcript and its summary, the plan, the opened agents, the jobs, any call waiting on a person, the inbox `cursor`, the beat count, a `stopped` flag and the *trace*: every invocation with its inputs, status and result. It is saved whole after every step (`AI:State:Save`), so a new process after a deploy, or another node after a failure, can load it and go on. A failed save costs one step, not the work.

5.2 What the model is shown

Each step sends one request, rebuilt from the state as in Figure 3.2, and an agent's details enter it only when the model asks. Each agent starts as one line; `open_agent` adds its functions as tools, with their input schemas and permission levels. A function above the chat's trust level is marked `NEEDS APPROVAL`, so the model knows before it chooses. Invoking an agent that is not open is refused, since the model would be guessing its inputs. At most eight agents stay open and at most 128 tools are sent; tools are only ever appended. A result over 16,000 characters enters as a preview plus its `storage_ref`, and `read` pages through the rest.

5.3 The step loop

One step, a *beat*, takes in events, calls the model once, carries out one action, records the result and saves the state. The action comes from a fixed set of twenty, among them `say`, `open_agent`, `invoke`, `start`, `spawn`, `plan`, `remember`, `schedule`, `sleep` and `finish`.

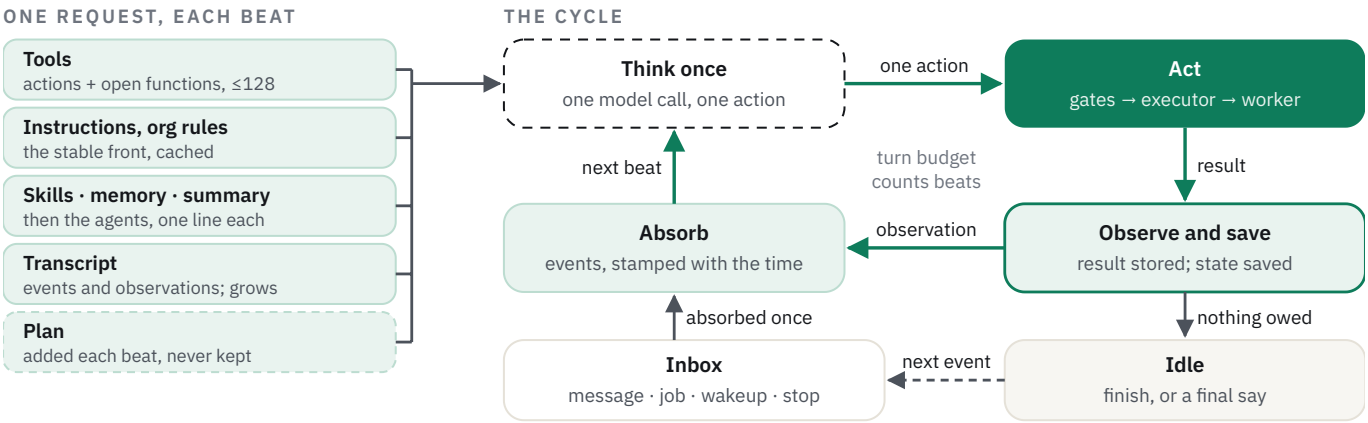


Figure 5.1 The step loop. The model (dashed) only proposes; every action passes the gates, and the state is saved before the next beat.

A malformed reply is corrected, never acted on. An invalid action is bounced back; two in a row cost no beat. Prose alone is delivered as a `say`. Of several actions, only the first runs. A `say` 92% alike to one already sent is refused, and a reply cut off at the output limit is dropped. A `finish` must give a reason the state bears out: `completed` is refused while plan items are owed, `awaiting_events` when nothing runs or is scheduled, `blocked` when nothing is blocked.

The turn budget stops runaway loops; it is not a quota. It counts beats since the person last wrote, up to the chat's `max_turns` (20 by default; 0 means no limit). At the limit the model is told to wrap up honestly and gets three more beats; then the runtime pauses the work, intact and resumable. A helper that runs out reports `budget`, not "done".

5.4 The plan, and nothing invented

The plan is the assistant's to-do list, shown to the person, and code decides when an item is proved done. It holds at most 12 items of 200 characters, each `pending`, `active`, `done` or `blocked`, with ids the runtime assigns. A blocked item must name its blocker; an item becomes active only when its dependencies are done. While an item is active, every successful call's storage ref and every finished job's id are attached to it as evidence. The model may name evidence too, but only refs the trace holds. An item done without evidence is shown *unverified*, since some work, such as answering a question, leaves no trace. Code enforces these rules; the prompt only describes them.

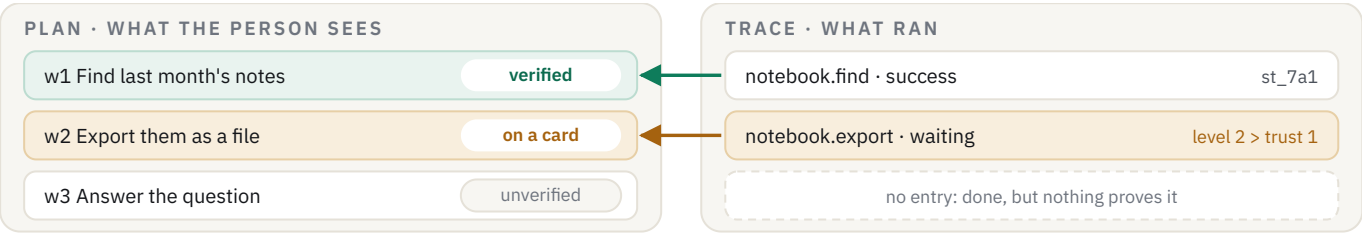


Figure 5.2 The plan and the trace. The words of each item are the model's; only a trace entry made while the item was active marks it verified.

> **I-5.1** A plan item is verified only by refs the trace produced. A `finish` that claims completion is refused while any item is pending or active.

No prompt can stop a model from inventing things, so the platform guarantees something narrower: *anything shown as data was read from a real result*. The words of a message are the model's; everything else on it is attached by the runtime from successful trace entries. A table or chart the `say` names (up to three) appears only if a successful call stored those rows, and is rendered from storage, so no value is retyped. Every write above level 0 adds a `success` part, and a file comes only from the call that produced it. Anything the runtime cannot back is dropped, and the model is told why.

> **I-5.2** Every table, chart, file and "verified" mark on a message comes from a successful trace entry and its stored result. A claim with nothing behind it is shown as the model's words alone.

5.5 Jobs and helpers

A call can also run in the background, and a bounded goal can go to a helper. `invoke` holds its beat until the call answers; `start` runs the same call as a *job*, through the same gates, whose result arrives later as a `job_done` event. A job whose process died is settled as an error, never retried on a guess.

`spawn` hands a goal to a *helper*, a job whose worker is a session of its own. The goal is its whole brief: it never sees the parent's transcript or plan, because paraphrased context is where invented values come from. It runs under the parent's contract, perhaps with fewer agents. At most three run per parent, and they do not nest: a helper cannot spawn, sleep, schedule, remember or save a skill. Its trace entries join the parent's, so its work is shown through the same evidence. Its job is `done` only if it finished `completed`; a success with no successful call leaves its items blocked.

5.6 Memory, skills, MCP servers, models

Memory. `remember` saves one lasting fact, visibly. The runtime can add memories but not delete them: a model that could forget could quietly rewrite what it knows about someone.

Skills are Markdown instructions, listed in the request and loaded with `use_skill`. A skill outlives its chat, so it is what an injected page would most like to plant. Only the person saves one: `save_skill` opens a card showing the skill whole, and the save is the person's own click.

MCP servers (Model Context Protocol tool servers) that a person adds appear beside the agents as `<server>.tools.<id>`. A server does not declare what its tools do, so each tool starts at level 3, and a new or changed tool comes back switched off. Address and credential are fetched only at call time (`Mcp:Server:Use`).

Models. A connection is a provider and a key, sent only as a `secret_ref`; the model is chosen per chat. Connectors cover OpenAI-compatible APIs, OpenAI Responses, Anthropic, Bedrock, Gemini and Vertex AI. When a chat's model changes, the new model is told the earlier actions were not its own.

5.7 Steering, stopping, sleeping

The person may write at any moment, and the assistant gives way at once. A half-written reply is dropped and asked for again, uncounted; a foreground call continues as a job; a call showing a screen receives the words too.

CONTROL	EFFECT
Stop generating	Between beats. A call the beat waits on ends at once; jobs are cancelled and given 15 seconds. The stop is saved, so a reloaded assistant stays stopped.
Stop everything in this chat	Pending cards are expired on the record; then the cycle, jobs, workers, helpers, browser and sleep end where they stand.
Stop all my work	`AI:Switch`: nothing of the person's is placed on a node, every chat is killed with ten seconds to record itself, and every runtime delegation for them is withdrawn, until they resume.
`sleep`	Waits at most a day for something outside, one per chat. A message is still heard at once; a stop cancels it.

5.8 The lease, and hydration after a restart

Two sessions never advance one assistant. On a node, a registry and a per-chat lock keep one session per chat. Across nodes, the numbered lease of §3.2 holds each chat on one node. A chat moves only after its lease lapses, under the next number, so a slow node never finds another acting for its chat.

Hydration rebuilds a live assistant from its saved state. It rewrites the request from the present and settles jobs left `running` as unknown (a helper resumes from its own state). It expires questions a dead process left, absorbs every inbox event past `cursor` exactly once, and resumes work left unfinished unless the chat was stopped. Approval cards stay open, because their parked calls are saved (§6.3).

> **I-5.3** At most one process acts for a chat at a time. A call that was running when its process died is reported as unknown and never re-run on its own.

CHAPTER 6

The gate

Every call the model proposes passes one checkpoint, the runtime's function executor. It compares the function's permission level with the chat's trust level. When the level is higher, the call waits for a person. Nothing the model writes is an input to that decision.

6.1 Two numbers and one rule

A function runs without asking when its permission level is at or below the chat's trust level, and asks first when it is above. Every manifest function declares a *permission level*: 0 read, 1 ordinary change, 2 wider change still inside the platform, 3 outside action. The level is a price, not a permission; a reviewer reads it beside the function before approving the agent. Each chat has a *trust level* on the same scale, chosen by the person: Ask for every change (0), Standard (1), Trusted (2), Autonomous (3). Grants decide which functions exist for a person; trust decides only how far work goes unasked. The default is 1, and a stored value outside 0–3 reads as 1, never 3, so damaged data never becomes the most permissive setting.

PERMISSION LEVEL	0 · ASK FOR EVERY CHANGE	1 · STANDARD	2 · TRUSTED	3 · AUTONOMOUS
0 · read	RUNS	RUNS	RUNS	RUNS
1 · ordinary change	ASKS	RUNS	RUNS	RUNS
2 · wider change	ASKS	ASKS	RUNS	RUNS
3 · outside action	ASKS	ASKS	ASKS	RUNS
Not granted, out of scope, invalid inputs, blocked site, unlisted package, host refused after a Deny	REFUSED	REFUSED	REFUSED	REFUSED

Figure 6.1 Permission level against trust level. *Asks* parks the call on an approval card. The last row does not depend on trust: the call is refused before any card, and the model is told why.

6.2 The order of the checks

The checks run in the fixed order of §3.6: manifest, grants, inputs, scopes, then level. Each failure returns to the model as an observation. A call that passes runs in its worker under its timeout, and is audited whatever the outcome. The contract is read again before every turn, so access withdrawn this morning is gone on the next one.

6.3 The approval card

An approval card asks a person to approve one exact call. It shows the agent, the function, its level and the *exact inputs* after defaults and references, so the person approves values, not a description. The card and the parked call both keep the action hash of §3.6. The call is parked before the card exists: the runtime saves the parked call, or the job set to `waiting_approval`, and only then emits `approval_requested`. If that save fails, the card is withdrawn and the call reports that nobody could be asked, never that someone refused.

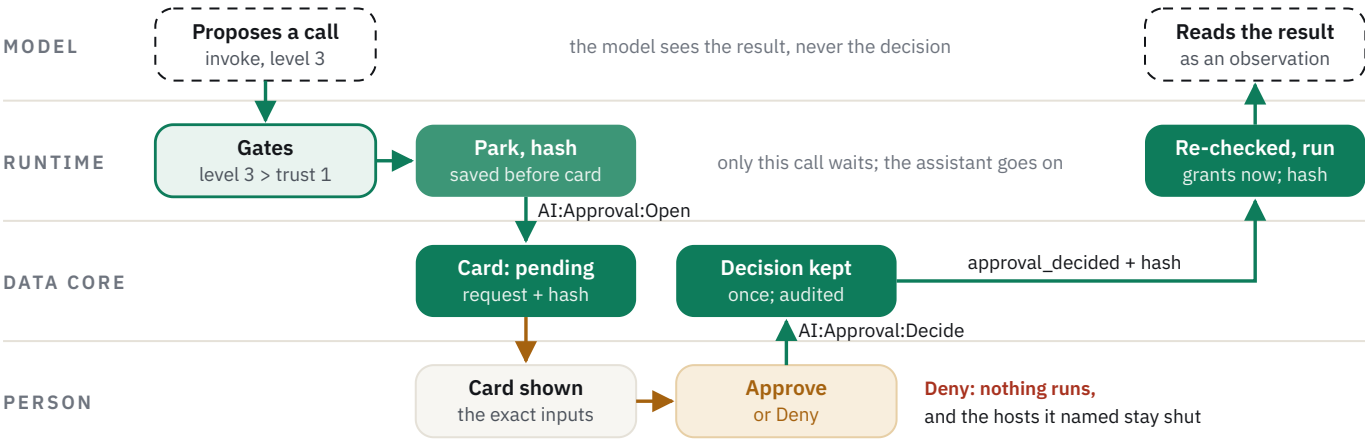


Figure 6.2 The life of an approval. The call is saved before the card exists, so a decision days later still resumes it; the Data Core records the decision before sending it. Amber steps wait for a person.

Only a person decides (§3.6). A decision lands only while the card is `pending`, so a second one gets `409 not_pending` and never overwrites the first. The runtime ignores a decision for a card its chat does not hold. An approval has no deadline: a parked job waits while the assistant works on, and a foreground `invoke` holds its beat until the card is answered or the person writes.

IF	THEN
the person approves	The level is not judged again, since the yes was to the call as shown; the grants are.
the person denies	Nothing runs. Until the person's next message, the hosts the call named are refused to every function of that agent, so a function that needs no card cannot make the visit instead.
the process died meanwhile	The card stays open. The decision resumes the saved call, with every gate run again and the hash recomputed; a mismatch blocks it.
a grant was narrowed meanwhile	The yes is refused with `not_permitted`.
the call was stopped or killed	The card is expired on the record and closed on the page.

> **I-6.1** What runs after an approval is exactly the call the card showed: its inputs hash to the value recorded when the card opened, and it passes the grants as they stand when the yes arrives.

> **I-6.2** A decision is recorded by the Data Core before it reaches the runtime, lands once, and can only be made by a person.

6.4 Questions, credentials and code

The same record, `ai_approvals`, carries every card where the platform needs an answer it cannot know. These cards have `kind: question` and differ in `expects`. Each waits at most a day, with the function's timeout paused, and lives only as long as the call that asked.

CARD	SHOWS	ANSWERED WITH
Question	an agent's question, up to 8 choices	a choice, words, or a file
Files	up to five files the person can see	the files ticked, each re-checked against what the person may see
Credential	entry, consent, choose or once; only the host, the site and field labels	values go straight to `Secrets:Credential:Save`, never to the card's record
Code	the whole code, its purpose, packages, hosts, credentials, files and review	allow or deny; allow opens the named hosts for that call only
Skill	title, summary, body as plain text	save or decline; a save creates it as the person

Code cards exist because the Browser agent runs scripts and the Code agent writes programs that nobody can list in advance. A package not on the organization's list is refused before anyone is asked. The chat's model then reviews the code, and the card shows its verdict: `agrees`, `differs` or `unread`. Code that would break the organization's rules never `agrees`. The review advises and never decides; one sent by an agent is discarded. An organization may let code the review agrees with run without a card. That is decided in the Data Core as the card opens, so the runtime cannot loosen itself, and the code is still announced in the chat with its first 25 lines.

6.5 The organization's Safety rules

Settings → **Safety** holds the administrator's rules for everyone. Code enforces each row at a fixed point:

RULE	ENFORCED
Sites no agent may open	at the egress proxy, for every worker and every manifest; while the list holds names, a host given as an IP address is refused
Scripts and programs	in the Data Core, as the card opens
Packages	in the runtime, before any card
MCP servers and inbound webhooks	in the Data Core; a blocked webhook answers 404
What a notification says	in the notifier

Notifications pass through services outside the deployment, so their content can be hidden: every push and email then says only "Something is waiting for your decision" or "A chat of yours has an answer", with a link. A person may choose the same but cannot undo the organization's choice.

Below the rows sit the organization's own rules for the assistant (`Settings:Guardrail`, at most 4,000 characters), placed in the request and in the code reviewer's instructions. They guide the model but enforce nothing; no code reads them. What must never happen belongs in the rows, the grants and the trust levels, which hold whatever any prompt says.

6.6 Why the model decides nothing

The model's output is shaped by everything it read, including text written by people other than the user, so nothing that grants authority reads it. The runtime acts under a one-hour delegation for one chat that opens a fixed list of actions: it can open and expire cards but not decide them, add memories but not delete them, read skills but not write them. A refusal reaches the model as an observation it can explain, and nothing it writes afterwards reaches a parked call's inputs.

> **I-6.3** No input to the gate is produced by the model. Levels, trust, grants and decisions come from the manifest, the chat's settings, policy and a person. The runtime can open a card but never answer one.

CHAPTER 7

Agents

An agent is a folder in a git repository: a manifest that declares what the agent may do, keep and reach, and the Python that does it. The platform reads the manifest without running the code, and an administrator approves one exact version of both. Every call is then held to that approved manifest, never to what the running code says about itself.

7.1 An agent is a package

An agent is self-contained: nothing about it is registered elsewhere. The folder holds the manifest, which is the contract; an entry point that returns the agent's tools, each with one asynchronous method per function; a README for the reviewer; and optional sample data. A function returns a result and a status, success or error. An exception becomes an error result.

At approval the folder is packed canonically, with sorted paths and zeroed times and owners. The **package digest** is the SHA-256 of the packed bytes, so one commit always gives one digest and any changed byte gives another. A package holds at most 5,000 files and 64 MB, and no symbolic link.

7.2 The manifest

The manifest is one YAML document with seven top-level blocks: `schema_version` (exactly `"1.0"`), `agent`, `implementation`, `network`, the optional `authorization` and `resources`, and `tools`. Figure 7.1 shows the lines of the Gmail agent's manifest that carry the design.

```yaml
agent:
  id: gmail                            # permanent: first segment of gmail.draft.send
  version: "0.3.2"                     # a label for people; the digest tells code apart
implementation:
  dependencies: ["requests>=2.31,<3"]  # built by the builder user, index only
network:
  hosts: [gmail.googleapis.com]        # the whole of where its code connects
resources:
  secrets:
    - id: google                       # a slot private to the install, not a credential
      oauth: {provider: google, scopes: [openid, …/gmail.modify, …]}
  data:
    - id: draft                        # kept as agt_…__draft, per install
      fields:
        - {name: status, type: select, storage: keys, options: [prepared, sent, …]}
        - {name: body,   type: object, storage: values}   # encrypted at rest
tools:
  - id: draft
    resources: {secrets: [google], data: [draft]}      # the tool's ceiling
    functions:
      - id: send
        permission_level: 3            # leaves the platform: a card below trust 3
        timeout_seconds: 60            # the host's clock; paused while a person is asked
        resources:
          secrets: {google: use}
          data:    {draft: [read, update]}              # no create, no delete
        inputs:
          properties:
            draft_ref: {type: string, x-resource: {type: data, id: draft}}
```

Figure 7.1 Part of the Gmail agent's manifest, condensed; comments added. The name `gmail.draft.send`, agent, tool and function ids joined, is what the model calls and what cards, grants and the audit trail record.

Four ideas carry most of the contract.

- **Resources appear twice.** A tool lists what its functions may touch at all; each function lists the operations it performs on each resource. A call gets only its own function's map, so `send` cannot delete a draft.
- **Fields are `keys` or `values`.** Keys are plaintext and filterable; values are encrypted at rest.
- **Hosts** are names, `*.name`, `name:port`, `{from_secret: slot.field}` for a host only the person knows, `[]`, or `any`.
- **Scopes** name a dimension a grant can narrow, such as Note's `notebook`. A function binds each scope to a top-level input with `from_input`.

FUNCTION KEY	WHAT IT OPENS	RULE
`permission_level`	Nothing; above the chat's trust level, a card	Required; 0 to 3
`timeout_seconds`	The host's clock for one call	Default 60
`authorization.scopes`	A grant may limit an input's values	Binds a top-level input
`llm`	`call.llm`, the chat's model	Not with `schedulable` or `watch`
`schedulable`	The clock may run it unattended	Level 0 or 1
`credentials`	`call.credential`, a login asked for as it works	Boolean
`code`	An allowed code card opens its hosts and `call.install`	Boolean
`watch`	Called, without a model, when the screen opens	Level 0

TABLE 7.1 What a function may declare. Each flag is opt-in and visible at review.

The permission level grants nothing; it decides whether a call needs a person's approval, and the reviewer judges whether it is honest. At the standard chat trust level of 1, levels 0 and 1 run and levels 2 and 3 raise a card.

7.3 Validation

One validator judges every manifest: when a source is read, at install, and when the runtime and each sandbox load it. It returns a list of errors, each with its path. The rules that shape the design:

- **JSON only.** Approval hashes the manifest as JSON, so YAML with no JSON equivalent, such as an unquoted date, is refused.
- **Bounds.** At most 16 tools of 25 functions, and 50 hosts, each a lowercase name, never an address or bare `*`. A manifest without a `network` block is refused.
- **Readable schemas.** Inputs and outputs use a closed subset of JSON Schema, without `$ref`, `anyOf` and their kin, because a schema a reviewer cannot read at a glance is one nobody checks.
- **Visible storage.** A secret or object field cannot be a key. A `from_secret` host must name a string kept in `keys`, so where an agent connects stays visible.

> I-7.1 Every component judges a manifest with the same validator and the same fingerprint function, so no two parts of the platform can disagree about what an agent declared.

7.4 Sources and catalogs

A **source** is a git repository an administrator saves on **Agents → Marketplace**. The Data Core fetches it at one commit, reads `decentai-agents.yaml` at its root, and validates every manifest listed there. **It never runs the code.** A catalog lists 1 to 100 entries, each just an id and a path. The whole catalog is refused if an id differs from its manifest's `agent.id`, a path leaves the repository, or anything is a symbolic link. A manifest that breaks a rule refuses only its own agent, which is listed with its reasons. A source belongs to the organization that saved it.

7.5 Review and approval

Approval binds one reviewed manifest and one set of bytes to an agent version. An administrator holding `agents:agent:install` approves the agent by name, and the install runs in order:

1. Fetch the commit the review showed; if the source was read again since, refuse with `source_moved`.

2. Validate again, and refuse a known version with a different manifest (`immutable_version`).
3. Write the approval row with `manifest_hash` , the SHA-256 of the manifest's canonical JSON.
4. Pack and store the folder per organization; on failure, restore the row.
5. Pin `package_digest` , grant the installer, audit `agent.installed` , and mark the row *preparing*.

No runtime is contacted; each node reports whether the code loads (§8.3). Archives are kept per organization, so one organization's compromise cannot reach another's code.

> **I-7.2** An approved version names exactly one manifest hash and one package digest. Nothing of an agent runs before an approval names it, and a version is never approved again with other content or other bytes.

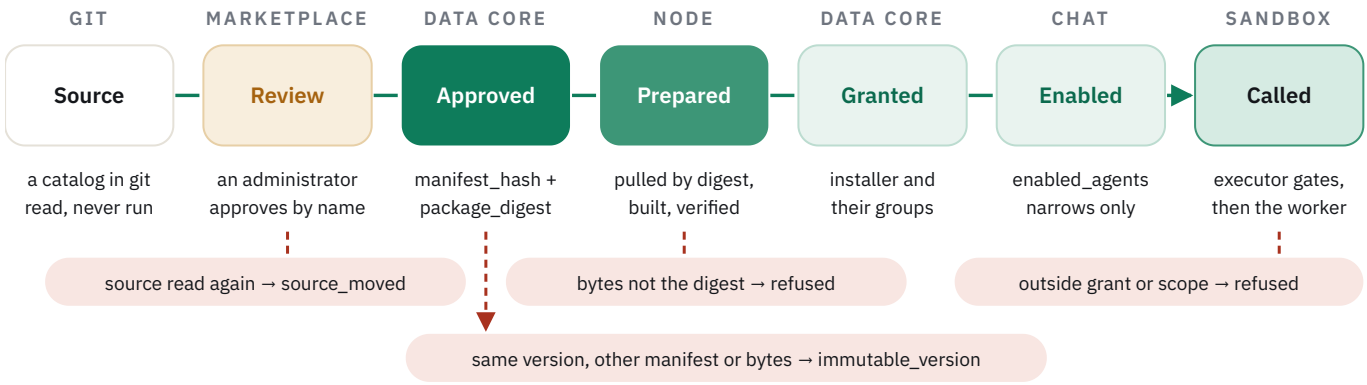


Figure 7.2 The lifecycle of an agent. Review waits on a person (amber); approval pins two fingerprints; each node pulls and verifies the bytes. Agent code (pale green) runs only after grants and the chat's list admit a call. Red marks each step's refusal.

7.6 Two names: refs and qualified ids

Inside the platform an agent is known by its **ref**, not by its author's id. At first install the platform mints the ref, `agt_` and twenty random characters, so nobody can predict one or derive another install's. Updates keep the ref, and with it the agent's records and grants. Grants, collections, secret slots, function names, cards and the audit trail all use the ref; only the manifest lookup and the worker call translate it back, so the code sees only `gmail` . Naming an agent after another gains it nothing.

For people and policies, each install also gets a **qualified id**: the author's id if it is free in the organization, otherwise `<source>__<id>` . It is assigned once and keys nothing.

7.7 Grants, scopes and the chat

A **grant** is a row, keyed by the agent, that says who may call which of its functions. Grants are not IAM policy statements, because functions come and go with installs. A grant names an owner (groups and users), functions (`"*"` or a list of `tool.function` names) and `constraints` , the values each scope may take. Grants only allow: without one there is no access.

The first install writes the smallest obviously right grant, the installer and the installer's groups; later access is set on the agent's **Access** tab. `"*"` follows the manifest as it grows, while a list keeps a new function closed until someone opens it. At call time the executor checks each scope's value, read from the input, against the grants; a constrained grant does not cover a call that gives no value. A saved credential likewise reaches an agent only through a revocable credential grant that a person writes.

A chat offers only what three sets share: the organization's installed agents, the grants that reach this person, and the chat's `enabled_agents` .

> **I-7.3** A person reaches a function only through a grant row on that agent and a chat that enables it. A chat's list can narrow what a person may use and can never widen it.

7.8 Versions and updates

A version never changes. Any change, a sample sheet included, needs a new `agent.version`, and taking an update is a new approval. Saved credentials carry over wherever the new version asks nothing new of their owners. At the next call the runtime sees the new digest and retires the old worker. A call still running there ends with an error and is not retried, because nobody can know what a half-finished call changed. A renamed `agent.id` is a different agent.

7.9 What agent code is given

Agent code reaches the platform only through `call`, the object the SDK, `decentai_sdk`, hands each function. The SDK is the only platform code an agent may import. `call` carries neither the person's identity nor a model key, and a refusal raises `ResourceDenied` with the reason in words.

`call.inputs`	Validated against the manifest.
`call.resources`	Records, files and secrets, checked against this function's operation map.
`call.progress, post`	A status line while it runs; a message to the person, at most three per call.
`call.ask(q, choices)`	A question card. The function's clock stops while it waits; unanswered after a day, it returns `None`.
`call.llm(…)`	The chat's model, through the platform. Needs `llm: true`.
`call.credential(host, fields)`	A login for a site not known beforehand, consented per agent and site; the model sees only labels.
`call.propose(code, purpose)`	Code on a card, beside the model's reading of it. With `code: true`, an allowed card opens its hosts for that call.
`call.screen`	A live picture the person can watch and take over (§8.11).

Without a chat, as in a test, `ask` and `propose` return `None` and `post` returns `False`, so the same code runs everywhere.

7.10 The catalogue

The project's own repository, `decentai-agents`, is an ordinary source of 38 agents, written against the same contract as anyone's.

AGENT	WHAT ITS MANIFEST SHOWS
Note	A `notebook` scope; all four levels, from `notes.find` (0) to `sync.push` (3).
Gmail, Outlook	An `oauth` account whose token stays in the Data Core; one host.
Mail	IMAP through the proxy, to a host named by a secret: `{from_secret: account.imap_host, port: 993}`.
Browser	`hosts: any`; `browse.run` at level 2 with `llm` and `credentials`; a level-0 `watch` function.
Code (`code_runner`)	`hosts: []` and `code: true`: the program reaches only what a person allowed on its card.
Tasks, Expenses	The model reads and code decides: every quote checked against the source.

TABLE 7.2 Agents worth reading for one pattern each.

Every Google agent asks for the same OAuth scopes, so one account serves them all. What keeps Gmail to mail is not its token, which could reach Drive, but that `gmail.googleapis.com` is the only host its worker may reach.

> **I-7.4** The runtime never imports agent code and never takes a worker's word for what it may do. Every resource, credential, host and model completion is granted host-side, per call, from the approved manifest's entry for the function now running.

CHAPTER 8

Execution nodes and the sandbox

The manifest decides what an agent is *given*. The sandbox bounds what its process can *take*, whatever its code tries. Agent code runs on an execution node, in a container that holds nothing else, as a user of its own, behind a file fence and a system-call filter, with one proxy as its only route out.

8.1 The execution node

An execution node is one AI Runtime and its Execution Sandboxes, deployed together as three Compose services. `ai-runtime` runs each chat's assistant, the executor and its gates, the agents' proxy and the sandbox registry. `sandbox`, the same image in another role, runs the **spawner**, which starts every worker and relays its traffic. `node-issuer` holds the node authority's private key, has no network, and writes the node a fresh credential into a volume the runtime reads. A node on its own machine dials a Data Core elsewhere and opens no port.

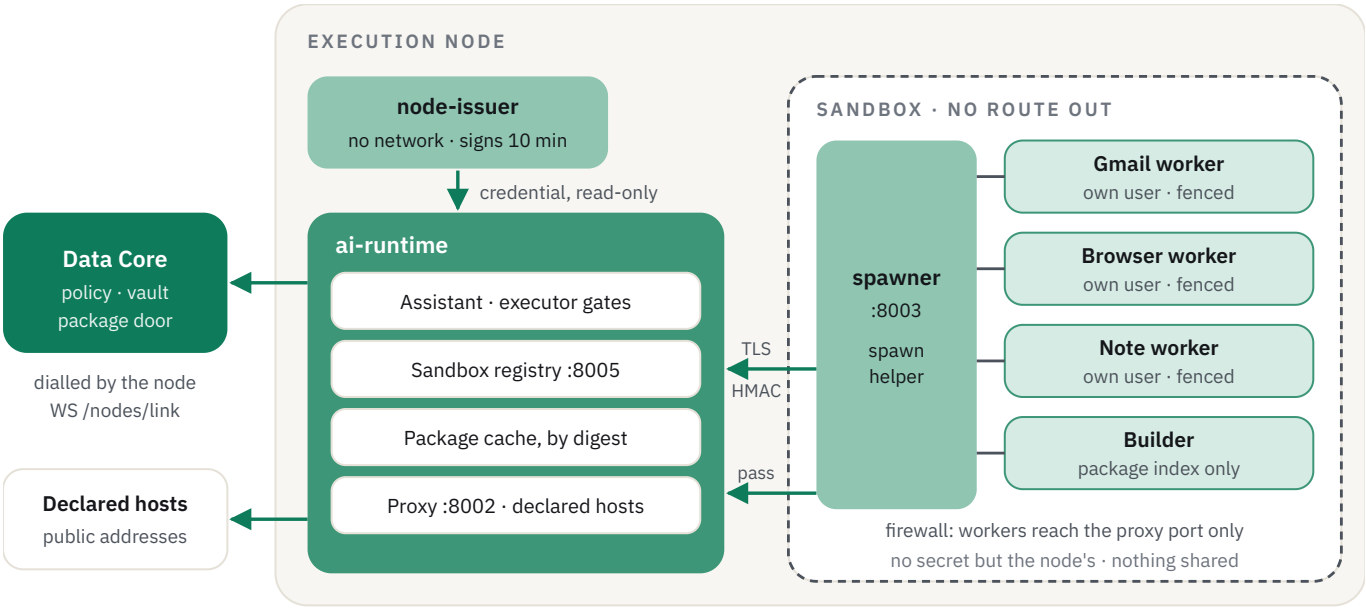


Figure 8.1 An execution node. Every connection is dialled outward. Workers' connections leave only through the runtime's proxy, to declared hosts. The dashed outline is the trust boundary: the sandbox's network has no route to the internet, the database or the Data Core.

8.2 How a node links to the Data Core

A node is admitted because a trusted authority signed its credential; nobody registers it. An administrator trusts an **authority** (a public key or a JWKS issuer) once, and every node it signs for is admitted. A node never holds the authority's key, and its credential lasts minutes, so a node taken over can sign for nobody and soon loses its name.

The node dials one WebSocket to `/nodes/link`. The Data Core holds no node's address, so a node can sit behind a firewall that admits nothing. Everything the Data Core sends is signed for that node and that request; a node treats anything else as silence, because a reply can carry a contract. A heartbeat every 20 seconds renews the node's chat leases, which last 60 seconds. Every call for a chat carries its lease number, so two nodes never act for one chat.

Code arrives through the **package door**, the only verbs a node may use: list the digests approvals pin, fetch bytes by digest, and report a package ready or failed. The runtime lists every ten seconds, so an agent moves from *Preparing* to *Ready* without waiting for a chat.

> **I-8.1** Nodes dial out and are admitted by authority, never by address. A node acts on nothing the Data Core's key did not sign for that node, and on no chat whose live lease it does not hold.

8.3 Environments built from approved digests

A node runs only bytes that match an approved digest and manifest hash. The runtime caches packages by digest and checks both before passing anything on, so tampered bytes fail for everyone instead of poisoning a shared copy. The runtime itself runs nothing from the cache.

Each sandbox fetches a missing digest from its runtime, checks it again, and unpacks its own copy. It builds an environment keyed by the dependencies and the interpreter, so an update that changes only code builds nothing. A **verification start**, one worker run as a user kept for it, proves the package loads. A build runs code too, so a separate **builder** user builds the wheels through a pass that opens only the Python package index.

> **I-8.2** Code reaches a sandbox only through its runtime, and is checked against the approved digest and manifest hash, both on the runtime and in the sandbox, before anything is written.

8.4 The worker pool

A **worker** is the process that runs one agent's code. The pool keeps one per approval, not per digest, so nothing an agent holds in memory crosses organizations. A worker starts on first call, stays warm for every chat of its organization, and keeps per-chat state, such as a browser, under `call.conversation`. It is disposable: replaced when its code changes or it crashes, ended when memory runs short. A new worker goes to a healthy sandbox with proven confinement, preferring one already running the agent, then one holding its package, then the least loaded.

8.5 The worker protocol

The **host**, the runtime's side of a call, and the worker exchange newline-delimited JSON, at most 2 MiB a line; a malformed line ends the worker. The worker's environment is an allow-list that never holds the model key, the Data Core's trust key or the runtime's settings. Every worker request names its `call_id`. The host answers it from that call's operation map, and only if that call was sent to this worker.

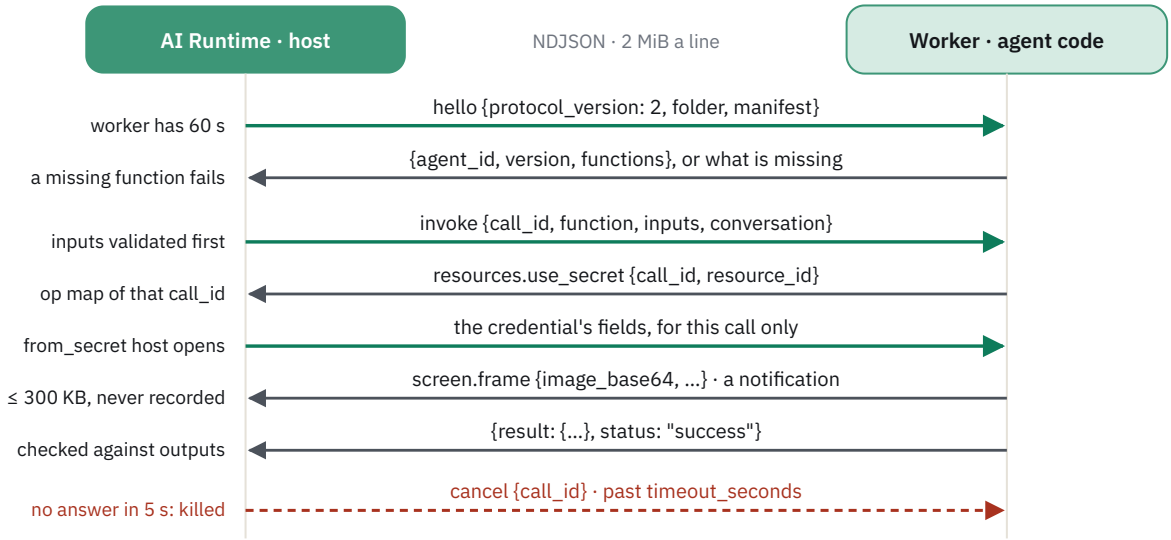


Figure 8.2 One call over the worker protocol. Green arrows are the host's, grey the worker's; the notes on the left are what the host checks at each message. The spawner relays every line unread. Killing a worker fails every call in flight on it, and none is retried.

8.6 The runtime–sandbox link

Only its own runtime can command a spawner. The two share one node secret, which Compose gives to nothing else, the Data Core included, and which never travels. Each side answers the other's challenge with an HMAC-SHA256 under the secret, prefixed with its role so that neither proof passes as the other's. Connections are TLS 1.3, and each proof covers the certificate fingerprint seen on that connection, so anyone in between fails. The sandbox removes the secret from its environment so no worker inherits it.

8.7 Confining a worker

Each worker runs as its own user, behind a file fence and a system-call filter. The **spawn helper**, a small compiled C program, does the one thing the platform's user cannot: start a child as another user. It starts only uids 20000–29999, and each approval has its own uid in each sandbox. It sets resource limits and no-new-privileges, becomes the agent's user, applies Landlock, and applies seccomp last. A worker's home is emptied before every start.

Landlock, the kernel's file and network fence, lets a worker read the system, its package and its environment, and write its home. On kernels that support it, the worker may open TCP connections only to the proxy port and cannot reach abstract sockets it did not create. **Seccomp** is a deny-list, because an allow-list would be a list of what somebody forgot. It refuses namespaces, mounts, `ptrace`, `bpf`, `io_uring` and similar calls. Each sandbox proves this confinement at start; the runtime places no worker where it failed, and an agent's page states what a weaker kernel cannot enforce.

8.8 Network confinement

Three layers leave the runtime's proxy as a worker's only way out. The sandbox's network is internal and reaches only the runtime. A firewall, written at container start by the one step that holds `NET_ADMIN`, lets worker users reach only the proxy port and refuses everything else, DNS included; that step then drops the capability for good. The spawner relays that port to the runtime's proxy, unread.

The proxy knows each worker by its **pass**, a token issued to that worker, and refuses any host the manifest does not name. It resolves the name itself, refuses an internal address, connects to the address it checked, and never opens TLS. Two kinds of host are lent for a single call: a `from_secret` host once the credential is handed over, and the hosts of a code card the person allowed. Every connection is logged.

> **I-8.3** A confined worker's only route off its machine is the runtime's proxy. The proxy admits only public addresses of hosts named by the approved manifest, by a credential handed to the running call, or by a card the person allowed.

8.9 Limits and timeouts

The platform sets these limits; no manifest can change them. CPU time is not limited, because the limit would count over a warm worker's whole life. The host's clock bounds each call instead.

BOUND	VALUE	ENFORCED BY
Processes, per agent user	2,048	the spawn helper
Open files / size of one file	4,096 / 1 GiB	the spawn helper
Memory and processors, all agents	`SANDBOX_MEMORY`, `SANDBOX_CPUS`	the container engine
Largest agent ended at	90% of the sandbox's memory	the spawner
Proxy connections	128 per worker, 1,024 in all	the proxy

TABLE 8.1 What a worker is held to.

CLOCK	VALUE	THEN
A function call	`timeout_seconds`, default 60 s	`cancel`; killed after a 5 s grace
Handshake / verification start	60 s / 120 s	worker killed / install fails
Dependency build	300 s	ended with everything it started
Sandbox heartbeat	2 s; gone after 30 s	its agents placed elsewhere
Node heartbeat / chat lease	20 s / 60 s	three missed beats and the chat moves

TABLE 8.2 The clocks, all owned by the host and never by the worker.

I-8.4 The host owns every clock. A worker cannot extend its own time, and a worker that does not answer a cancel is killed with everything its user runs.

8.10 Draining, many sandboxes, many nodes

One runtime serves many sandboxes; another needs only the node secret and the runtime's address. `Nodes:Node:Drain` stops new workers on one sandbox or all, while running workers finish. Labels on a node's credential gate nothing, because what a node says of itself is reported, not believed.

The Data Core sends a chat to the node holding its lease, otherwise to the organization's own nodes first, then the chat's last node, then the least loaded. With `placement: own_only`, nothing of the organization reaches a shared node. A chat that moves is rebuilt from the record, and a call running on the old node is reported unknown, never rerun.

8.11 The live screen

An agent that drives a screen, such as a browser, can show it live through `call.screen` and let the person take over. Each frame, a JPEG or PNG of at most 300 KB, passes from worker to host to Data Core to page; it is never recorded, and dropped rather than queued. Input travels back the same way, after the Data Core checks that the person may write in the chat, and reaches only the call that showed the screen. **Take over** pauses the run; **Hand back** resumes it and tells the agent to look again. Opening the screen calls the agent's `watch` function without the model, which is why that function must be level 0. Nothing of the browser is reachable from the person's machine.

WHAT THE SANDBOX DOES NOT DO

It does not separate the people of one organization: a worker serves every chat of its approval, and a host lent to one call is open to calls beside it. It does not stop an agent misusing what it was rightly given. Memory is shared among agents and enforced by ending the largest. Before kernel 6.12, two agents that both intend to can share an abstract socket. The container, sharing the host's kernel, remains the outer wall.

CHAPTER 9

Secrets and data

Everything a person owns lives in one database and is shared by one mechanism. Secrets go further: their values reach an agent's code for one call, and never a person or the model.

9.1 The data layer

The Data Core keeps all its state in one MongoDB database of **45 collections**, each declared with its validator and indexes in one schema. At start it creates what is missing, enforces every validator and drops any undeclared index, so the database is exactly what the code says.

The collections cover identity and access (12), chats (8), agents (5), the data layer (6), connections (3), execution nodes (4), reach (5) and evidence (2). Thirty-three carry `org_id`; the rest hang off a row that does, or belong to the deployment rather than to a tenant.

The six data-layer domains share one document shape. `resource_ref` names the document, `resource_id` is a category label many documents share, and `owner` maps the groups and users who may see it. `keys` holds plaintext metadata that can be searched; `values` holds the encrypted payload, which cannot. A document holds at most 50 entries and 10 MiB of values.

Visibility is a filter, not a check. Every read builds its query from the caller's organization, the domain and the owner map, so it cannot match a document the caller may not see. A check after a lookup can be forgotten; a filter inside the lookup has to be removed on purpose. Even the *Everyone* share resolves within the reader's own organization.

Two rules govern writes. Nobody may share beyond their own groups and the peers in them, and only the creator, the *steward*, may edit, re-share or delete. A per-domain escape grant lifts both rules, and stewardship can be transferred. One sharing engine applies this to secrets, records, files and skills (`PERSONAL`) and to model connections and agent sources (`INFRASTRUCTURE`).

9.2 Encryption at rest

Every encrypted field is sealed with AES-256-GCM, using a 32-byte key and a fresh 12-byte nonce, and records its key version. The document's `_id` is the associated data, so a ciphertext copied into another row fails to open. It binds to the id rather than the owner, because owners change.

Keys come from `SECRET_ENCRYPTION_KEYS`, a list of `version:base64-key` pairs. `SECRET_ENCRYPTION_ACTIVE` names the version for new writes, by default the highest. Mixed version widths (`1`, `2`, `10`) are refused, because versions compare as text and a stale key would become active. Without keys the Data Core does not boot.

Rotation is three steps: add a version and restart; re-encrypt everything, after a `--check` pass that only counts; retire the old key. The re-encryption tool asks each store for its own encrypted fields, because a field a hand-kept list misses is lost for good when its key goes.

A value whose key is missing shows as unreadable, so one bad row does not blank a page. At use it fails loudly, with a 409 that says how to fix it: an agent acting on an empty credential is worse than one that stops.

> **I-9.1** A sealed value opens only in the document it was written for, under a key version the deployment still holds. Nothing falls back to a default key or returns partial plaintext.

9.3 Credentials

A credential is a secret saved against a **definition**, which lists its fields and says which are encrypted or required. Definitions are derived from manifests, each in a namespace private to one agent (`agt_<ref>__<slot>`), so a category names exactly one approval. If names were shared, any repository could declare the same fields and read what was stored under them.

Definition versions are immutable. After an agent update, a credential moves to the new version only if that version asks nothing new of its owner. Otherwise it is marked *outdated* and refused at use, with a sentence saying what is missing.

The creator is a credential's steward. Sharing lets colleagues' chats *use* it, not edit it. A **grant** points one saved secret at an agent's declared slot of exactly the same shape, so several agents share one key and a rotation stays one act. At use, the Data Core takes the chat's binding, else a grant to that agent, else the person's default, else the only visible instance. If several remain, it reports an ambiguity rather than guessing.

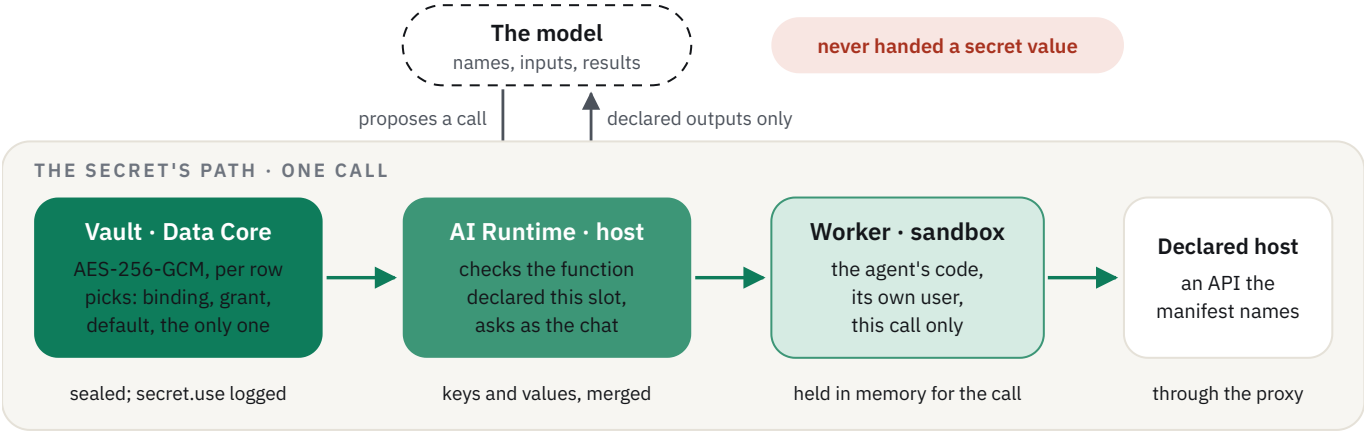


Figure 9.1 The path of a secret. Only a runtime principal may call `Secrets:Secret:Use`, and the values reach only a worker whose function declared the slot. The model sees only the declared outputs, after validation.

A worker starts with no credentials. When a function asks for its slot, the host checks the declaration and calls the Data Core as the chat's delegation. The Data Core decrypts, writes `secret.use` to the audit trail, and returns keys and values together, because a password is of little use without its hostname. The values reach only the worker's memory, and the proxy opens the credential's host for that one call.

One gap remains: a warm worker could keep a value between calls. An access token's expiry bounds that risk; nothing bounds it for a typed key. This is why agents are read before they are approved.

> **I-9.2** A secret's values leave the Data Core only through two runtime-only operations, `Secrets:Secret:Use` and `Secrets:Credential:Resolve`, and only toward the worker of a function that declared the slot. No read by a person returns them, and the model is never on the path.

9.4 Connected apps

Some credentials come from an OAuth sign-in, and the platform, not the agent, runs that flow. A definition's `oauth` block names the provider, its addresses and its scopes. The organization holds one client **registration** per provider, with a write-only client secret, and the registration pins the provider's addresses. An agent that names other addresses cannot connect, because the client secret and the person's tokens would reach its server.

`Secrets:Oauth:Start` writes a single-use state row, valid ten minutes, holding a PKCE verifier and the browser session that asked. The provider returns to `/oauth/callback`, which spends the state, requires the same person in the same session, exchanges the code, and creates or refreshes the credential.

The web client's session cannot ride in a popup, because the client runs on its own origin. The callback therefore parks the code behind a two-minute, one-use hand-off, posted only to the `PUBLIC_APP_URL` origin. The web client or phone app spends it with `Secrets:Oauth:Complete` under its own session, where the same-person check runs.

At use, the Data Core first refreshes an expiring token. The agent receives the account and an access token; the refresh token never leaves the Data Core. If the person revokes access at the provider, the credential is marked `needs_reconnect`.

9.5 Records and files

Records (`agents_data`) are what an agent keeps for a person, such as a watcher's memory of what it has seen. Each follows a shape the manifest declares, which a person's writes must also meet. A person may write only where the manifest's `user_access` allows, and may otherwise read and delete. Exports leave out fields marked secret.

An agent reaches records and files only in its declared slots; anything else answers as nonexistent. The exception is a ref the call was *handed*, in its inputs or the person's answer, which it may read but not write or delete.

Files hold bytes as well as a document. Only an upload creates one, because a storage location taken from a caller could point anywhere. The storage id is derived from the bytes, name, purpose and uploader, and the bytes go with the last document that refers to them. Uploads are capped by `MAX_UPLOAD_MB` (default 25).

`FILE_STORAGE_PROVIDER` selects `local` disk, `s3` (or any S3-compatible store), `azure_blob` or `gcs`. Each document records the connector that wrote it, so a store the deployment has left can still be read. Each connector refuses a path that climbs out of its root. Memories, chats and execution nodes sit outside this layer on purpose.

9.6 What leaves a deployment

Nothing goes to DecentAI's makers. Data leaves only by these routes, each decided by a named role. A chat's messages go to its model provider, and with its model key and granted credentials to the execution node that holds it; *Own nodes only* keeps an organization off shared nodes. An agent's traffic goes through the proxy to its declared hosts. Sign-ins go to OAuth providers, MCP calls to a person's own servers, notifications through push services and the mail server, and outbound webhooks to subscribed addresses. Speech goes to a provider only where an administrator chose one. The vault, the trail, files, records and chats stay in the deployment's own storage.

CHAPTER 10

Evidence, monitoring and audit

The model's words are its own. Whatever the platform asserts about a turn comes from records it wrote: what the chat saw, what was allowed and done, what the processes did, and what the models used.

10.1 The activity stream of a turn

A chat keeps two logs in `ai_chat_events`. The **inbox** holds what the assistant must absorb: messages, finished jobs, fires and webhooks. Each event is numbered and written before the assistant hears it, so a session rebuilt after a crash resumes from its bookmark. The **outbox** is the audience's replayable log of frames such as `message_created` and `activity`.

An `activity` line has a kind, such as `call_started` or `call_finished` (with status and duration), and the lines of one call share its `call_id`. The Data Core validates each event as it records it; a malformed one is lost from the replay, not from the live stream. Screen frames are never recorded.

10.2 The audit trail

The audit trail, `ai_audit`, is append-only and written at the Data Core's chokepoints. Each line holds the event type, who acted for whom, the chat, function and resources, details and time. The Data Core records its own acts, such as `secret.use`, `llm.use`, approvals, grants, `schedule.*` and `work.stopped`. Any other gateway action that changes state, refused or not, becomes `platform.action` with its outcome: `success`, `denied` or `failed`.

The runtime may write exactly one type, `execution`: the agent, function, levels, an outline of the inputs, the outcome, the duration and the hosts the proxy counted. With only that type, the runtime cannot forge an approval or a credential read.

The trail is a witness, not a gate. An append is tried twice; a line the database refuses goes whole to the error log at `CRITICAL`, and the act still happens.

A person reads their own trail (`ai:audit:list`), also shown per chat; the organization's whole trail needs `ai:audit:list_all` . Lines outlive their actors. No secret value is written, but an agent's error text and the model's proposed inputs are kept as they came.

> **I-10.1** Nothing through the gateway edits or deletes an audit line. The runtime adds only `execution` lines; every other type is the Data Core's record of its own act.

10.3 The evidence behind a claim

Evidence (chapter 5) rests on these records. While a plan item is active, the storage ref of each successful call and the id of each finished job attach to it, and the model may cite only those. An item marked done without them shows as *unverified*. Tables and charts are rendered from the rows stored in `ai_chat_storage` , so no value is retyped.

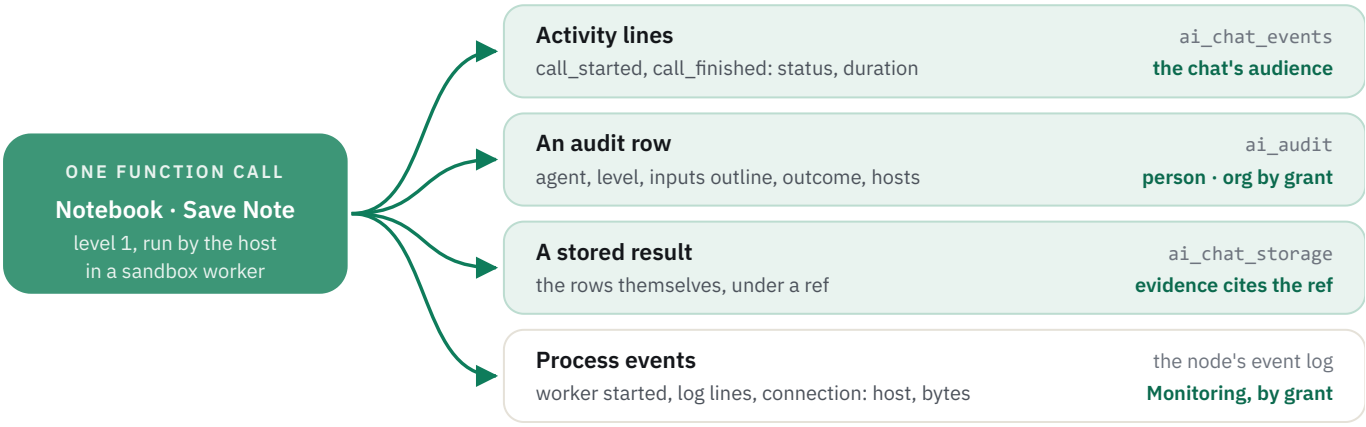


Figure 10.1 What one call leaves behind. Tinted rows are Data Core records; the white row is the node's own log, read over the node link. A call that asks the chat's model also adds an `llm_usage` row.

10.4 Monitoring

Monitoring shows what the agents' processes did. The runtime and each Execution Sandbox write JSON lines to their own folders, because no container may read another's. The lines record workers starting and ending, logs, processes, memory and connections. A connection line holds the host and resolved address, whether it was allowed and reached, the bytes each way and the duration, but never contents: the proxy does not open TLS.

Each writer keeps at most 64 MiB, and a failed write never fails what it records. The Data Core asks each connected node a `monitor` question over its link and shows each organization only its own agents' lines.

VIEW	GRANT	WHAT IT SHOWS
Overview	`agents:monitor:summary`	the agents' acts and model calls in a window, from the audit trail
Tokens	`agents:monitor:tokens`	tokens, cache share and estimated cost by model, purpose, agent and person
Agents	`agents:monitor:usage`	limits; each running agent's memory, processor, processes and disk, every 5 s
Events · Connections	`agents:monitor:usage`	every line, filterable; connections with their bytes

No member holds these grants by default. What an agent did is what every person did through it, so seeing it is a decision someone makes.

10.5 Token accounting

Every model answer a chat receives becomes one `llm_usage` row, kept for 396 days, with its purpose, connection, model, tokens and duration. The organization, person and chat come from the delegation, never from the runtime.

Providers count cache tokens differently, so the runtime normalises every answer into `input` (uncached), `cache_read` , `cache_write` , `output` (thinking included) and `reasoning` . The cache share, `cache_read / (input + cache_read + cache_write)` , measures how well prompt caching works.

Rows are sent in batches of at most fifty, which no beat waits on. Cost is an estimate at the catalog's prices; an unpriced model's tokens appear as `unpriced_share`, never as a cost.

10.6 Package checks and health

A sandbox proves a package before it runs it. It checks the bytes against their digest and the manifest against the approved hash, builds the dependencies, then starts one worker whose greeting proves the package loads: the **verification start**. The Agents page shows *Preparing* → *Ready* or *Failed*, and a failure is remembered until the approvals change.

Health has three routes. The Data Core's `/healthz` checks liveness only. Its `/readyz` pings MongoDB and answers 503 without naming the database, because anyone may call it. The runtime's only HTTP route is `/internal/health/live` on port 8001.

CHAPTER 11

Time and reach

A chat keeps working while nobody watches. A clock wakes it, outside systems may call it, and when something needs a person the platform finds them. Every path has a limit, and one switch stops them all.

11.1 Schedules

A schedule (a row in `ai_schedules`) wakes a chat at a set time, in one of two modes. A third kind of wake belongs to the assistant alone:

- A **wake** delivers a note, such as "morning brief", and the assistant decides what the moment means.
- An **invoke** runs one function the manifest marks `schedulable`, at level 0 or 1, with fixed inputs and no approvals or model. It wakes the assistant only when the result's `wake_field` is non-empty, so an empty five-minute sweep costs no model call.
- A **sleep** is the assistant's own one-off wake: one per chat, at most a day ahead, not shown on the Schedules page. A longer wait must be a schedule the person can see and stop.

A time is given as `at` (ISO 8601, in the person's time zone), `delay_seconds`, `every_seconds` (at least 60) or a five-field `cron`. A chat holds at most 50 rows.

The clock is the Data Core's, so a row fires once however many nodes hold its chat, and even when none does. Every five seconds the clock places each due chat on a node. It *claims* the row with a conditional write that holds only if the due time is unchanged, then sends a signed `fire` over the node link. Four timing rules follow:

- A period counts from when the row was due, not when it ran, so an hourly row stays on the hour.
- An overdue row fires *once* and rejoins its schedule. A Data Core that was down replays no backlog.
- The next run comes at least 60 seconds after the last fire, so a miscomputed row cannot spend a model turn on every tick.
- A row never runs twice at once. One fire may wait; later ones fold into it as `merged`, and a pause drops it as `skipped`.

> **I-11.1** The Data Core fires each due row once, under a conditional claim. No node decides when, and no outage becomes a burst of missed runs.

11.2 Webhooks in

An inbound webhook lets an outside system wake one chat with typed data. Each webhook (`whk_…`, at most 25 per person) has a name, the person's instruction, up to 40 typed fields and one **standing chat**, which every event wakes and which keeps its plan between events. Its secret (`dws_…`) is shown once. It is *sealed, not hashed*, because checking an HMAC needs the secret itself.

```
POST /hooks/<webhook id>
Authorization: Bearer <the webhook's secret>
X-DecentAI-Timestamp: 1760518000                # with a signature
X-DecentAI-Signature: sha256=<HMAC-SHA256 of "<timestamp>.<body>">
Idempotency-Key: <the caller's id for this event>   # optional

{"customer": "Acme", "amount": 1200, "invoice_id": "INV-1043"}
```

#	CHECK, IN ORDER	REFUSED WITH
1	webhook on; its person active and not stopped; organization allows webhooks	`404` one answer for all
2	body at most 64 KB	`413`
3	bearer secret, in constant time; signature where required or sent; timestamp within 5 min	`401`
4	at most 60 events a minute, counted per Data Core process	`429`
5	a JSON object, declared fields only, each of its type	`400` naming the field
6	idempotency key seen within a day	`202` first id, `repeated`

An accepted event is answered **202** at once, so a caller never waits on a model. Each call past the first check is kept for 30 days in `webhook_deliveries` and recorded on the trail by field names, never values. The node holding the chat then receives the event, with the instruction read from the webhook's own record. The assistant sees the person's instruction as theirs, and the posted fields as an `EVENT` marked *data, never instructions*.

> **I-11.2** What an outside system posts reaches the assistant as data beside the person's own instruction, never as an instruction.

11.3 Webhooks out

An outbound webhook tells a person's own system that something happened in a chat. A person may subscribe up to ten `https` addresses to `approval.waiting`, `reply.ready`, `run.finished` and `run.failed`; card and reply events are queued only while the chat is closed. A body carries the event, the chat, a line, a link and the time, and the reply's text only on request. It never carries a secret or the conversation.

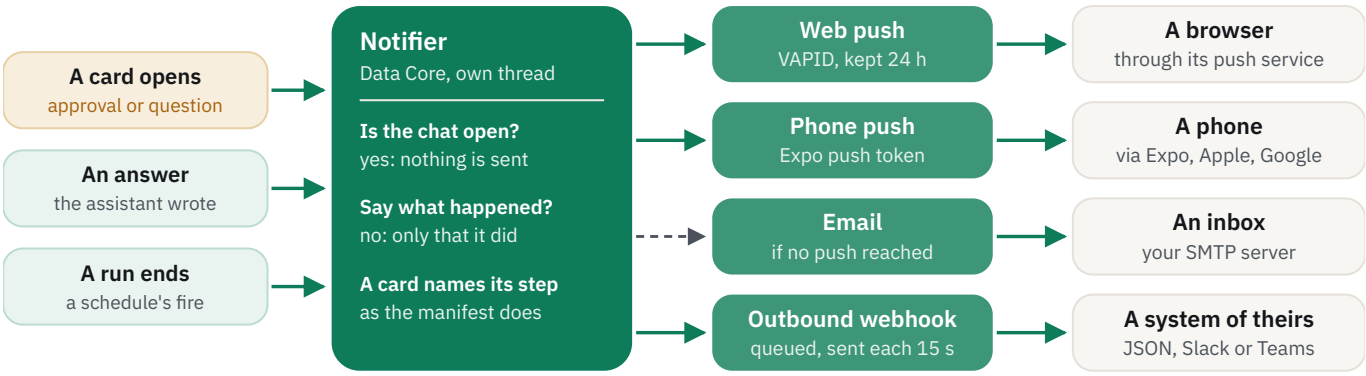
JSON bodies are signed as inbound calls are, with the subscription's sealed `dss_…` secret; Slack and Teams receivers check no signature, so their bodies are unsigned. Each attempt has a 10-second timeout and follows no redirect. A timeout, a 5xx, a 408 or a 429 is retried with a growing delay, from one minute to six hours, for one day. Twenty failures in a row pause the subscription.

Every address is checked when it is subscribed and again before every send, and refused if any DNS answer is off the public internet, including IPv4 carried inside IPv6. Anyone may subscribe, so a notification must not become a route to the database or a cloud's metadata address. The connection itself is made by name, so DNS rebinding between check and connection remains a known limit.

11.4 Notifications

The notifier (chapter 3) pushes to every subscribed device when a card opens or the assistant answers in a chat no browser of the person has open, and emails only if no push reached anyone. It runs on its own thread, so no turn waits on a push service. Browsers get web push signed with the deployment's VAPID key pair, which a push service holds for up to 24 hours; phones get Expo push. A device the service reports gone is dropped.

Email goes at most once per chat every ten minutes and answer pushes at least two minutes apart; cards are not spaced, because someone is waiting. A card names the agent and step from the approved manifest. Because the text passes through outside services, a person may turn *Say what happened* off, and an organization may require that of everyone; the notification then says only that something is waiting.



Each carries a line and a link into the chat. Never a secret, never the conversation.

Figure 11.1 The paths that reach a person. Email (dashed) goes only when no push reached anyone. Every push endpoint and outbound address passes the address check first.

> **I-11.3** No push and no outbound delivery goes to an address that resolves off the public internet. The check runs when the address is registered and again on every send.

11.5 Stop everything

The stop at the top of every page (`AI:Switch`) ends all of a person's work without relying on anything to agree. It acts in a fixed order:

1. The person is marked stopped. From then on nothing of theirs is placed on a node and no key is minted, whether a message, a schedule or a webhook asks, and their webhooks answer 404.
2. Every chat a node may be serving for them is killed with `stop` and `force` , with ten seconds to record what it did.
3. Every delegation of theirs is deleted, so a process that ignored its end can read no credential, key, record or file.

The stop is recorded as `work.stopped` . Resuming (`work.resumed`) is the person's own act and places nothing on a node: a chat returns when it is opened or a schedule falls due, and a schedule due during the stop fires once. No runtime can lift the stop, so stopped work cannot restart itself. Stopping one chat is narrower: it ends what is running but sets no lasting stop, and its schedules keep firing.

> **I-11.4** While a person is stopped, no chat of theirs is placed on a node and no delegation is minted for them. Work already running loses its keys.

CHAPTER 12

Clients

The engine is headless: it serves an API and no page. Three clients use it — a web client, a phone app and a desktop app — and none of them holds authority. Each shows what the Data Core says the signed-in person may see, and asks the Data Core for everything else.

12.1 One API

A client renders a small, fixed set of routes on the Data Core's one published port. `POST /app` carries every action, named `Domain:Controller:action` and checked against the caller's policies (chapter 4). `WS /chats/{chat_id}` carries one live chat. The other routes sign people in, move files and take redirects and webhooks. No client has a route or a permission of its own.

The reason is the threat model. A client runs on a device the platform does not control, and a modified copy can send any request it likes. A client that hides a button does so for convenience, not for safety.

At sign-in the web client sends `client: "web"` and the phone app `client: "native"`. Each receives the session token in the response body and presents it as a Bearer header. A browser on the engine's own origin that names neither is given a cookie instead.

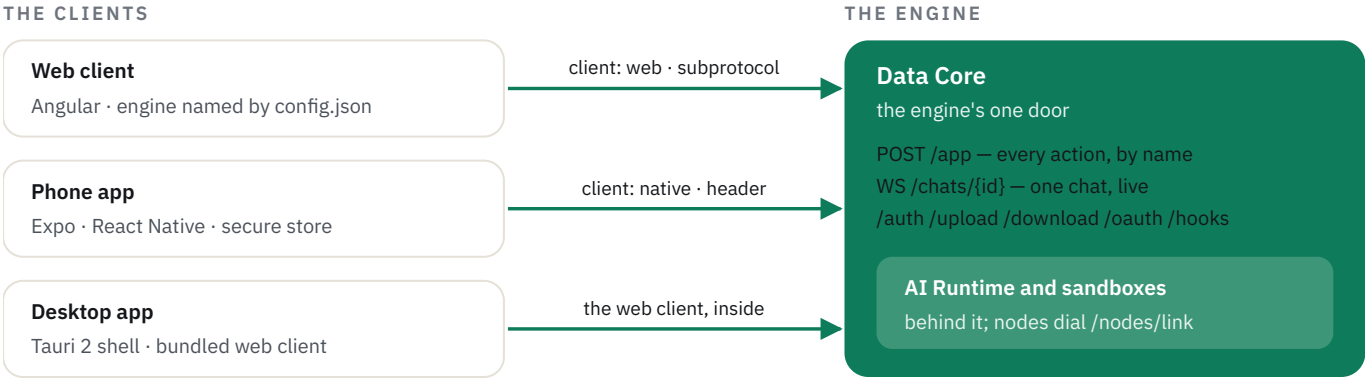


Figure 12.1 — Three clients, one API. Every client speaks only to the Data Core, across origins, with a session token it holds itself. The desktop app is the web client in a window of its own, not a fourth protocol. No client reaches the runtime, a node or the database.

	WEB CLIENT	PHONE APP	DESKTOP APP
Engine address	person's choice, else `assets/config.json`, else the page's origin	typed on the first screen; kept in the secure store	the shell answers `assets/config.json`
Token kept in	`localStorage` or `sessionStorage`, bound to its engine	the platform's secure store (the keychain on iOS)	as the web client, in the app's web view
Chat socket	subprotocol `decentai.bearer, <token>`	`Authorization` header on the handshake	as the web client
Speech	the computer's own voices and recognizer where local; else the server's	the phone's recognizer and voices, by default	as the web client
Notifications	web push (VAPID)	Expo's push service	none of its own
Scope	everything: chats, agents, settings, admin	chats, cards, schedules, stop-everything	everything, plus running the engine locally

> **I-12.1** No client is a security boundary. The Data Core filters every read it serves a client inside the query, and resolves every action a client sends again from the session on the server. A modified client gains nothing an honest one lacks.

12.2 The web client

The web client is one build of static files, pointed at an engine by one file read at run time. It is an Angular 20 single-page application, served by its own container, by any static host, or from inside the desktop app. The file, `assets/config.json`, says `{"engine": "https://api.example.com"}`; the container writes it from `ENGINE_URL` as it starts. The service worker never caches it, so changing it moves every browser on its next load.

An address the person entered comes first, then `config.json`. Failing both, the client tries the page's own origin, for a reverse proxy that serves the engine beside the app. It accepts that origin only if `GET /auth/setup` answers JSON carrying `setup_required`, which only an engine does; otherwise the Connect screen asks.

The token is stored with the engine that issued it and dropped when the address changes, so one engine's session is never offered to another. A browser cannot set headers on a WebSocket handshake, and a token in the URL would be written into every log on the way. So the chat socket offers the token in its subprotocol list, after the name `decentai.bearer`. Because the engine is on another origin, the browser enforces `CORS_ALLOW_ORIGINS`: the Data Core answers preflights only for the origins listed there.

The chat page

A chat is shown as **turns**: a person's message and what came of it. Socket events carry sequence numbers. On reconnecting, the page replays what it missed from the Data Core's event log and holds live frames until the replay is done, so no event is applied twice or out of order. If the log has moved past where the page stopped, the page reloads the chat instead.

A turn that did work has a **work block**: the assistant's plan, each item with its status and what the trace proved for it (chapter 5), and the calls made, grouped by agent. **Cards** appear in the thread where work waits (chapter 6), and each is answered once. **Read-aloud** speaks a reply in pieces, so the first sentence plays while the rest is prepared.

12.3 The phone app

The phone app is a remote control: it keeps a person in the loop away from a desk. It is an Expo (SDK 57) application in React Native 0.86 for iOS and Android, and connects to any DecentAI server by address. It covers chats, a **Cards** list of everything waiting for the person, schedules, background work, notification settings and the switch that stops everything of theirs. Agents, credentials and organization settings are set up in a browser.

The first screen takes a server address, an email and a password. Signing out deletes the token and keeps the address. React Native, unlike a browser, lets a socket carry headers, so the chat socket sends the token as an `Authorization` header. Push notifications go through Expo's push service: the app registers its push token with `Settings:Notifications:Subscribe`, and tapping a notification opens its chat. The phone must reach the server: `localhost` on a phone is the phone, so a server on a person's own computer needs a tunnel or a network address.

12.4 The desktop app

The desktop app is the released web client in a window of its own, pointed at a server or at an engine it runs locally. It is a Tauri 2 shell for Windows and macOS. On first opening it asks once: **connect to a server**, or **run DecentAI on this computer**. Either way the shell itself answers `assets/config.json`, with the server's address or `http://localhost:<port>`.

The page's origin is `http://tauri.localhost` on Windows and `tauri://localhost` on macOS. No web page can claim those origins, which is why the engine's default `CORS_ALLOW_ORIGINS` lists them. The page gets none of the shell's native commands, so nothing rendered in a chat can stop or uninstall the engine. Links other than sign-in windows open in the person's browser.

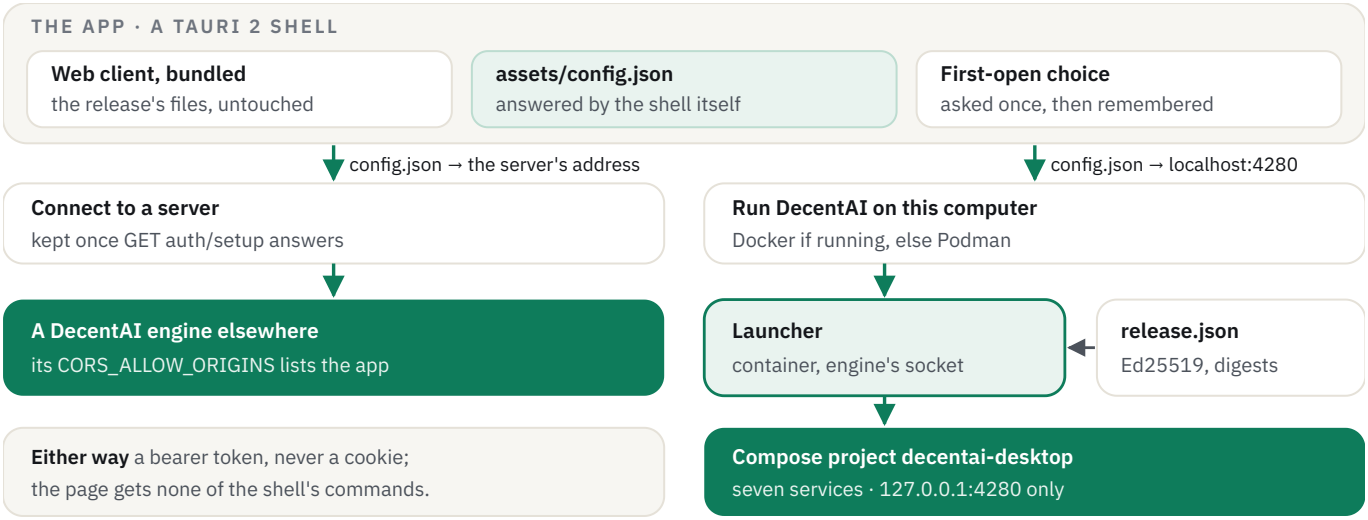


Figure 12.2 — The desktop app's two modes. One shell and one bundled web client; only the engine that `config.json` names differs. In the second mode a launcher container, handed the container engine's socket, starts the engine's released images once the release's signature verifies.

Running DecentAI here

The app installs nothing itself: a **launcher** container does the work, and trusts only a signed release. The launcher carries its own `docker compose` and is handed the container engine's socket — Docker where it is running, otherwise Podman — so it behaves the same on Windows and macOS and is updated by pulling an image. It starts the engine's seven services as Compose project `decentai-desktop`, apart from a developer's own `decentai` stack, from images pulled by digest. Keys are made once. A first run that stopped half way reuses them, because keys made twice leave a database nobody can read. The settings are those of section 13.4.

A release is `release.json` (schema 2.0) with an Ed25519 signature beside it. It names by digest the engine's images, `mongo`, the launcher and the bundled web client, and the oldest launcher that understands it. The app reads it to choose the launcher, and the launcher reads it again before installing. Both believe it only if it is signed by one of the keys the launcher ships with; an unsigned file from one's own build is installed only when explicitly allowed. An update pulls every image first, copies the database, then switches releases. If the new stack is not healthy, the previous release returns with the copy restored.

> **I-12.2** The desktop app runs only images named by digest in a release file whose Ed25519 signature verifies against a key it ships with. Otherwise, whoever could put a release file in front of an install could put their own software on the machine.

CHAPTER 13

Deployment

The platform is one image per process, configured entirely by environment variables: a laptop, a virtual machine and a cloud differ in values, never in code. TLS, public hostnames and proxies belong to a deployment, and the recipes in `decentai-deploy` supply them.

13.1 What is deployed

The engine is three images and MongoDB 7, and it publishes one port. The images are the Data Core's, which also runs the node issuer; the AI Runtime's, which also runs the Execution Sandbox; and speech. Together they make seven containers on the three networks of chapter 2. The one published port is the Data Core's, plain HTTP, `4280` by default. The web client is served separately.

An execution node *dials* the Data Core's `/nodes/link` over the public address people use, and nothing dials a node (chapter 8). So moving agent code to another machine opens no inbound port; that machine only needs to reach the engine's HTTPS address.

13.2 Topologies

TOPOLOGY	RECIPE	WHAT RUNS WHERE	PUBLISHED
One computer, from source	the engine's setup script	the seven containers; the web client from its own checkout	`127.0.0.1:4280`
The desktop install	the desktop app's launcher	project `decentai-desktop`; the web client inside the app	`127.0.0.1:4280`
One server	`single-host`, `ROLE=all`	everything, behind Caddy	Caddy: 80, 443
Core and a separate node	`single-host` with `ROLE=core`; `node`	core: Caddy, web client, Data Core, Mongo, speech. Node: runtime, sandbox, node issuer	core: 80, 443; node: nothing
AWS, two VMs	`cloud-vm/aws`, `layout = "two-vm"`	the core recipe on one EC2 VM, the node recipe on another	core: 80, 443; node: nothing
AWS, one VM	`cloud-vm/aws`, `layout = "one-vm"`	`single-host` with everything	80, 443

single-host needs a Linux host with Docker Engine and Compose 2.24 or later, two DNS names (`api.<domain>` and `app.<domain>`) and ports 80 and 443 open; Caddy handles the certificates. Its setup script runs inside the Data Core's image, so the host needs no Python. It makes every key once, points the public addresses at the two names, lists the app and the desktop origins in `CORS_ALLOW_ORIGINS`, and believes forwarded headers from Caddy alone. Two names keep the engine's origin apart from the app's, as the engine is tested; `LAYOUT=path` serves both under one name.

With `ROLE=core` the server runs the Data Core's side and writes a **node bundle** once: the Data Core's public key and address, the node's sandbox secret and the node authority's private key. The **node** recipe on the other machine unpacks the bundle and deletes it, and it is deleted on the core too, so the authority's private key exists on the node alone. The node publishes no port; once linked, it appears on the Data Core's Nodes page.

13.3 Two VMs on AWS

`cloud-vm/aws` is Terraform for the whole deployment: a VPC with one public subnet, two EC2 VMs (Ubuntu 24.04, encrypted disks), an Elastic IP for the core, Route 53 records when a hosted zone is named, and daily disk snapshots, the last seven kept. The core admits 80 and 443; the node admits nothing. SSH is off by default, and the VMs are reached through Session Manager.

The VMs need no git: each first boot copies its recipe from a private S3 bucket and runs it. The node bundle travels once through an encrypted SSM parameter and is then deleted. Both VMs require IMDSv2 with a hop limit of one, so no container can reach the VM's instance role. With `ecr_mirror = true`, images not yet public are pulled from ECR in the account.

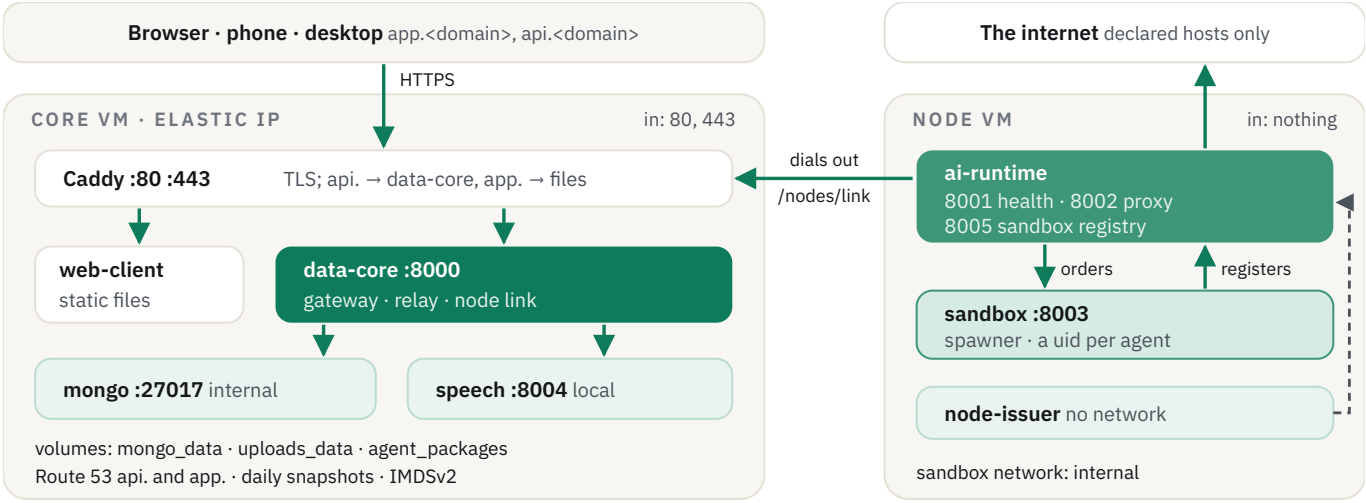


Figure 13.1 — The two-VM topology. People reach only Caddy on the core VM. The node VM accepts no connection: its runtime dials the Data Core over HTTPS, its sandbox registers on 8005 and takes orders on 8003, and agents reach the internet only through the proxy on 8002. The node issuer, on no network, rewrites the node's credential into a shared volume (dashed). The node VM holds no database, no uploaded file and none of the Data Core's keys.

> **I-13.1** A deployment publishes one door: the Data Core's port, or the TLS terminator in front of it. The database is on an internal network that only the Data Core and the seeder reach. An execution node publishes nothing, because it dials the Data Core and is never dialed.

13.4 The desktop's local install

The desktop install (section 12.4) is the one-computer topology with every edge turned inward. The Data Core listens on `127.0.0.1` only, with nothing in front of it, and `CORS_ALLOW_ORIGINS` holds the app's two origins alone. No setup code is asked (`SETUP_CODE=none`), since nobody else can reach the address, and sign-up is closed. No email is sent: invitations become temporary passwords shown once, and the launcher resets a forgotten password. Speech fetches no model until an administrator turns it on. Reaching the install from a phone takes a tunnel the person sets up.

13.5 The configuration surface

Each process receives only the secrets it needs. The settings file, `config/deploy.env`, goes whole to the Data Core and the seeder. The node's sandbox secret goes only to the runtime and its sandboxes, and the authority's private key only to the node issuer. The runtime never sees the database password, the encryption keys or the Data Core's private key. The settings that matter most:

SETTING	DEFAULT	WHAT IT DECIDES
`PUBLIC_ENGINE_URL`	`http://localhost:4280`	the engine's public address; the base of OAuth redirects and webhook addresses
`PUBLIC_APP_URL`	`http://localhost:4300`	the web client's address, for links in mail and notifications; the only origin the OAuth popup reports to
`CORS_ALLOW_ORIGINS`	`localhost:4300`, `localhost:4200`, the two desktop origins	every origin the web client is served from. Never a wildcard: any origin listed can act for a signed-in person
`FORWARDED_ALLOW_IPS`	`127.0.0.1`	whose `X-Forwarded-For` is believed: the TLS terminator only, never `*`. Session records and the sign-in lockout depend on it
`TOKEN_SECRET_KEY`	required	signs session tokens. Missing, the template's value, or under 32 characters: the process refuses to start
`SECRET_ENCRYPTION_KEYS`	required	versioned AES-256-GCM keys as `version:key`, all versions the same width
`FILE_STORAGE_PROVIDER`	`local`	`local`, `s3`, `azure_blob` or `gcs`; each file is read from the store its record names
`SMTP_HOST`, `MAIL_FROM`	unset	outbound mail. Unset, people are added with a temporary password shown once
`SETUP_CODE`	empty	empty makes a first-run code and logs it, so whoever finds the address first cannot claim it

Keys are generated, never chosen, and no generator writes its output to disk. Encryption keys rotate by appending a version: new writes use it, and a re-encryption tool rewrites stored values until the old version can be dropped. A ceiling on trust levels is deliberately not a setting: trust belongs to a chat, bounded by what its person may do.

> **I-13.2** A database is readable only with the keys it was written with. Setup and update scripts make each key once and keep it thereafter. They add a key a newer version needs, and never replace one that exists.

13.6 Volumes and backups

A deployment is its settings, its database, and its file and package volumes; everything else is rebuilt.

`mongo_data`	the database: identities, chats, records, encrypted values. Back up.
`uploads_data`	file bytes, with the `local` store. Back up.
`agent_packages`	approved agent packages, the only copy the platform controls. Back up.
the settings	the keys: without them stored credentials are unreadable and the node authority is lost. Back up.
the rest	caches, speech models and the node credential, all rebuilt or fetched again.

`single-host`'s backup script writes all four, taking a `mongodump` archive while the database runs. On AWS the disk snapshots come in addition, not instead.

13.7 Updates

A release is a version tag on a commit whose tests passed. The release workflow builds the three images for Intel and ARM, publishes them to `ghcr.io/decentai-io` and names each digest in a GitHub release. The web client is released separately. Recipes pin both versions and pull; they never build.

An update runs the new seeder first, while the old code still serves, then rolls the services. A seed adds collections and actions and drops no data, so older code runs against a newer database and going back is naming the previous version. As it starts, each version removes indexes its schema does not declare, and policy actions its catalog no

longer has. A node whose protocol differs from its Data Core's is refused, so a core and its nodes are updated together.

CHAPTER 14

Verification

Each property this paper states is meant to be checked, not believed. The checks are automated suites that use the platform as a person, an agent or an attacker would; a hostile agent run in the deployed containers; and tests that hold the documentation to the code.

14.1 The suites

The counts are test functions at the time of writing; parametrization runs more. They grow with every fix, so the counts of 15 October 2026 (section 14.5) are smaller.

SUITE	TESTS	RUNS AGAINST	EXERCISES
AI Runtime	1,233	an in-memory simulator of the Data Core's services; no database	the assistant loop, executor gates, approvals, clock, stops, proxy, evidence, the documentation
Data Core	1,125	MongoDB 7	identity, policy invariants, the delegation fence, data layer and encryption, OAuth, webhooks, nodes, audit
Spanning	190	both, a real runtime process	agent install and manifest, scope enforcement, the chat contract, the full stack
Confinement, live	97 of the runtime's	the runtime's and two sandboxes' containers on Linux	a hostile agent's escapes, and the spawner across containers
Speech	20	the speech image; 3 with the real models	the audio protocol, the model catalogue's digests
Web client	about 670 cases	Karma and Jasmine, headless Chrome	services, chat state and connection, cards, components
Desktop launcher	95	pytest	release verification, installation, updates and roll-back
Desktop shell	10	`cargo test`	the first-open choice, the release file, the window

Continuous integration runs on every pull request: the runtime suite with the documentation checks; the Data Core and spanning suites against `mongo:7`; the confinement suites in containers; and speech. The phone app is held to the TypeScript compiler and Expo's linter, and the deploy repository to static checks that need no cloud account.

14.2 A scripted model

Runtime tests run real agent code in real worker processes against a **scripted model**, which returns replies written in advance. A test must give the same result on every run, which a real model does not, and the claims under test concern what the platform does with *whatever* the model proposes. The simulator answers the Data Core's services in memory; agent authors use it too. The spanning suite closes the gap the simulator leaves. In its full-stack test a person's socket reaches the Data Core, which attaches a real runtime process, and the runtime calls back over `/app` under the chat's delegation. Only the model is stubbed.

14.3 An agent written to escape

The live suites install an agent that is hostile on purpose. Each of its functions attempts one escape: read the runtime's settings, write into its own package, signal the runtime, connect past the proxy, resolve a name directly, open an unused socket family, `ptrace`, create a namespace. A test passes only if the attempt is refused. Beside each refusal a positive control checks that ordinary work still functions, since a sandbox that refused everything would also pass. A second hostile package attacks at build time.

> **I-14.1** A green confinement run has run the hostile agent. The live suites skip everywhere but the container stack, and the CI job fails when either passed nothing. So a run in which every test was skipped for want of a kernel feature cannot be reported as a pass.

14.4 Documentation held to the code

Five tests keep the documentation, from which this paper is distilled, in step with the code. Every relative link reaches a page and a heading on it; every repository path a page names exists; every documented setting is read by some code, and every example setting is documented; and the published list of actions is exactly what the Data Core's catalog holds today. These tests check what a page *points at*. Whether a sentence is true remains for a reader to verify.

14.5 The reported evaluation

The long paper (October 2026) reports these suites, as test functions, from runs on 15 October 2026. Its "Backend" is the Data Core.

SUITE	TESTS	EXERCISES
Runtime	1,031	assistant loop, executor gates, approvals, clock, stops, proxy, evidence, documentation
Backend	924	identity, policy invariants, delegation fence, data layer and encryption, OAuth, webhooks, audit
Cross-component	180	approval-to-package path; end-to-end chat across backend and runtime
Example agent	24	an agent run as its author would
Total	**2,159**	
of which sandbox, live	83	hostile agent in the deployed two-container topology

On the main branch that day (Python 3.11, MongoDB 7) every suite passed with no failures. The runtime had 1,056 passed and 85 skipped: the 83 live sandbox tests, run separately, and two others. The backend had 969 passed and 6 skipped; cross-component 216 passed and 3 skipped; the example agent 24 passed. The live suites then ran in the deployed two-container topology on Linux 6.18 (WSL2), which offers every Landlock feature the sandbox uses. All 83 passed, none skipped: 58 confinement tests and 25 of the spawner across containers, in 88 seconds.

The same month, the whole code base was reviewed line by line. Every case found to make the code fail, hang or answer wrongly was fixed together with a test that repeats it. Design questions the review raised and did not settle were published instead (chapter 15).

14.6 What is not evaluated

- **Model behaviour.** No test calls a real model. The tests show that a proposal cannot pass the gates; they do not measure whether a model resists injected text or chooses well.
- **Performance and scale.** No overhead is measured, nor behaviour with many organizations, chats or concurrent schedules.
- **The kernel.** Confinement results hold on the kernel tested; a kernel flaw is a way past every layer.
- **The clients.** The web client's suite was not part of the reported evaluation, and the phone app has none.

CHAPTER 15

Limits and open questions

What the platform does not do or guarantee, in one place. Each limit is **chosen** (a trade-off made on purpose), **open** (raised by the October 2026 review and not yet decided) or **inherited** (from what the platform runs on).

15.1 What is not guaranteed

AREA	LIMIT	KIND
The model	The platform does not make the model correct. Injected text still steers whatever runs without a card: every granted function at or below the chat's trust level (at level 3, all of them). There is deliberately no administrator ceiling on the level.	chosen
Plans	A plan item counts as verified when it has *some* evidence: one unrelated successful call can attach to every active item, and the model chooses which reference it cites (RSN-10).	open
Approved agents	During a call an agent holds the decrypted credential and may talk to its declared hosts; the proxy sees hosts and byte counts, never content. A warm worker could keep a credential beyond the call. Review before approval is the control.	chosen
Declared hosts	A declared host is in effect a declared address: once connected, an agent can ask that address for another site it serves (SBX-6).	open
Agents together	Two colluding agents can pass bytes through `/tmp` or `/dev/shm` (SBX-9), or through an abstract socket before Linux 6.12. Colleagues granted the same agent share one worker process.	open; inherited; chosen
The kernel	All containers share it. Some locks need a recent kernel or rights the operator grants; without them the platform runs and says so. The system-call filter exists on x86-64 and arm64 only, and is on or off for every agent at once.	inherited
Resources	Memory is a shared pool sampled once a second; disk use is counted but not capped. Tenants share the agents' container, and an agent writing enough to its error output can push others' connection records out of the bounded log (LIB-7).	chosen; open
A compromised node	It can act for the people whose delegations it holds, within the 41-action fence, for up to an hour, and without opening cards, because a scheduled chat must act while its person is away. It cannot decide a card, read the database or open a sealed value.	chosen
The audit trail	Append-only by the code's shape, not by mathematics: with no hash chain, someone holding the database password could alter it undetectably. A call's inputs are recorded in outline as the model proposed them, so a password typed into a chat can reach the trail (EXE-11).	chosen; open
Secrets in content	Results are validated for shape, not scanned for secrets: a credential an agent returns reaches the model and the trail. The platform does not seal file bytes; they rest on its volume or in a bucket the provider encrypts.	chosen
Webhooks	Only DecentAI's own signature scheme is verified. An outbound address is checked by name and connected to by name, which leaves a DNS-rebinding window. An event a chat could not receive is not retried.	open
Failover	A chat whose node stops answering waits for its lease to lapse (60 seconds) before it moves. A call that was running there is reported as unknown and never rerun by itself.	chosen
The Data Core	It is trusted with everything: a compromised Data Core is a compromised deployment.	chosen

15.2 What is not built yet

No release has been published at the time of writing. The `decentai-deploy` recipes pull released images, so until a release exists they run images of one's own (or the ECR mirror, on AWS). Recipes for Azure, Google Cloud, Kubernetes and marketplaces are planned and absent. The desktop install listens on `127.0.0.1` only, sends no email, and does not carry over installs from the earlier monolithic app. The phone app configures only its own use.

15.3 Open questions

The October 2026 review raised design questions it did not settle: choices, not defects, where either answer may be right. They are published with the code as a list, each naming the function it concerns and whether it was reproduced or found by reading. The long paper counts 137; the list holds 130 at the time of writing, as rows are deleted when decided. A sample a deployer should know:

ID	QUESTION
EXE-12	Every string in a call's inputs is treated as a reference the function may read, whether or not the input is marked as one. In the simulator one agent read another's record when the reference was in its inputs.
EXE-14	There is no ceiling on a function's `timeout_seconds`, on how often a function asks the model, or on how many cards one call opens.
ASST-14	Any object with an `"action"` key inside a prose reply is run as an action, so a quoted example can be executed.
SBX-7	A credential field holding `.com` is lent as a host pattern, and a leading dot means every host under it.
LIB-5	A declared dependency may still be a direct URL, a `git+` address or a path, which the builder fetches.
EDGE-9	Tool arguments that do not parse become no arguments, silently, in every model connector.

Publishing these is deliberate. A security design that lists only what works asks to be taken on faith; one that lists where it is undecided lets a reader weigh the rest.

APPENDIX

Glossary

The words this paper uses for one exact thing.

Action — one operation the gateway dispatches, named `Domain:Controller:action`; what a policy grants.

Agent — an approved program: a manifest and code in a git repository, installed as one exact commit.

AI Runtime — runs each chat's assistant, the executor every call passes, and the egress proxy. It has no database.

Audit trail — the append-only record of what the platform allowed: calls, approvals, credential reads, installs, grants, stops.

Card — something in a chat that waits for a person: an approval of one exact call, a question, code to allow, a login.

Data Core — the engine's one door: identity, policy, the vault, the data layer and the relay; the only part that reaches the database.

Delegation — a short-lived credential minted per chat and bound to one node, under which the runtime acts for the person within 41 actions.

Evidence — tables, charts and files the platform shows beside a reply, rendered from stored results the trace proves exist.

Execution node — one AI Runtime and its Execution Sandboxes, admitted by an authority's key. It dials the Data Core and is never dialed.

Execution Sandbox — a container where agents' workers run, started by its spawner; it reaches only its runtime.

Function — one operation an agent's manifest declares, with input and output schemas and a permission level.

Gateway — `POST /app`, the one route every action goes through and where permission is checked.

Grant — permission to use something: a policy allowing an action, an agent granted to a person or group, a credential lent to an agent.

Launcher — the desktop app's container that installs and updates the engine on one computer from a signed `release.json`.

Lease — which node holds an active chat, numbered and renewed by heartbeat; a call with an old number is refused.

Manifest — an agent's contract: its functions and levels, records and files, credential, hosts and dependencies.

Permission level — 0 to 3 on a function: a read, an ordinary change, a wider change, an outside action (one that leaves the platform or cannot be undone).

Plan item — one entry in the assistant's plan, kept by the platform with its status and the references offered as proof.

Proxy — the runtime's egress proxy on port 8002: a worker's only way out, open to the hosts its agent declared.

Record — what an agent keeps for a person, in its manifest's shape, with plaintext keys and encrypted values.

Skill — a short instruction an organization wrote for the assistant, pulled into a chat when the task calls for it.

Trace — what the assistant actually ran in a chat; evidence and plan proofs are checked against it.

Trust level — 0 to 3 on a chat, set by its person: a function at or below it runs unasked; above it, it waits for a card.

Worker — the process an approved agent's code runs in, as its own user, fenced to its own files.